TALENTI AUDIT PACKET

As-Built Audit Packet

This consolidated packet compiles the current architecture, deployment, operational, data, API, frontend, user workflow, and decision-record documentation into a single auditor-ready PDF. Mermaid diagrams are rendered into the document and placed on dedicated pages for print readability.

Generated
19 April 2026 at 07:45 pm

Document Count
10 compiled source documents

Rendering Model
A4 packet with dedicated rendered diagram pages

Purpose
All-in-one as-built audit and architecture evidence packet

The packet is assembled from repository documentation and preserves the source language of each contributing document while presenting it as a single printable artifact.

Table of Contents

Platform Architecture Overview

`documentation/ARCHITECTURE_OVERVIEW.md`

1. Document Purpose and Scope
2. System Overview
   - Service Inventory
3. Business Purpose
4. Architectural Principles
5. System Context
   - External Actors
   - System Boundary
6. High-Level Architecture
   - Key Architectural Boundaries
7. Component Architecture
   - 7.1 Frontend SPA
   - 7.2 Backend API
   - 7.3 Backend Worker
   - 7.4 Model Service 1 — Culture/Behavioural Fit
   - 7.5 Model Service 2 — Skills/Technical Fit
   - 7.6 Python ACS Service
8. Frontend Architecture
   - 8.1 Routing Structure
   - 8.2 API Client Pattern
   - 8.3 State Management
   - 8.4 Browser-Direct Integrations
9. Backend Architecture
   - 9.1 Application Structure
   - 9.2 Middleware
   - 9.3 Database Migrations
   - 9.4 Key Runtime Flows
10. Data Architecture
   - 10.1 Database
   - 10.2 Core Entity Groups
   - 10.3 File Storage
   - 10.4 Multi-Tenancy Model
11. Integration Architecture
   - 11.1 Azure Service Integrations
   - 11.2 Inter-Service Communication
12. Authentication and Authorization
   - 12.1 Authentication Mechanism
   - 12.2 Authentication Flows
   - 12.3 Authorization Model
   - 12.4 Internal Service Authentication
13. Security Architecture and Controls
   - 13.1 Implemented Controls
   - 13.2 Security Considerations and Gaps
14. Configuration and Secrets Management
   - 14.1 Configuration Approach
   - 14.2 Key Environment Variables
   - 14.3 Secrets in Deployed Environments
   - 14.4 Local Development
15. Environments and Deployment Architecture
   - 15.1 Environment Matrix
   - 15.2 Azure Resource Inventory per Environment
   - 15.3 Container App Configuration
16. CI/CD and Release Approach
   - 16.1 Pipeline Overview
   - 16.2 PR Quality Gates
   - 16.3 Immutable Image Pipeline
   - 16.4 Release and Promotion
   - 16.5 Infrastructure as Code
17. Logging, Monitoring, and Operational Support
   - 17.1 Logging
   - 17.2 Monitoring
   - 17.3 Alerting Baseline
18. Error Handling and Resilience Considerations
   - 18.1 Implemented Patterns
   - 18.2 Resilience Gaps
19. Architecture Decision Records
   - ADR-001: FastAPI + PostgreSQL Backend
   - ADR-002: JWT Authentication with Local User Table
   - ADR-003: Azure-First Cloud Services
   - ADR-004: REST API with Versioned Prefixes
   - ADR-005: Database-Backed Background Jobs
   - ADR-006: Dual Scorecard Architecture
   - ADR-007: Decision-Dominant Scoring Logic
   - ADR-008: Python ACS Service Separation
   - ADR-009: Trunk-Based Release with Immutable Promotion
20. Known Limitations, Technical Debt, and Risks

20.1 Technical Debt
20.2 Architectural Risks
20.3 Known Limitations
21. Assumptions and Unresolved Questions
Assumptions
Unresolved Questions
Appendix A: Repository Evidence Summary
Directory Structure
Key Configuration Files
Test Coverage
Appendix B: Audit Findings — Contradictions and Stale Documentation
B.1 Contradictions Between Documentation and Implementation
B.2 Stale or Incomplete Documentation
B.3 Documentation Gaps
Appendix C: Gap Register for Auditors
C.1 Documentation Gaps
C.2 Implementation Gaps
C.3 Security/Control Gaps
C.4 Operational Gaps
C.5 Governance Gaps

Architecture Diagrams

`documentation/ARCHITECTURE_DIAGRAM.md`

Source Of Truth
1. High-Level System Design
System Design Notes
2. Runtime Topology
Runtime Notes
3. Environment And Infrastructure Topology
Environment Notes
4. Delivery And Promotion Architecture
Pipeline Notes
Summary

Release Pipeline

`documentation/RELEASE_PIPELINE.md`

Branching and Merge Rules
Workflows
Release Contract
GitHub Environment Variables and Secrets
Promotion and Rollback
Local Pre-Commit Gate
Environment Protection Notes

Deployment Runbook

`documentation/DEPLOYMENT_DEV_V2.md`

Topology
Database
Required Backend App Settings
Required Backend Worker App Settings
Required ACS Worker App Settings
CI/CD
Pinned Model Promotion Runbook
GitHub Environment Variables
Migration

Monitoring and Observability

`documentation/MONITORING.md`

What to Monitor
API Health
Database (PostgreSQL)
Azure Services
Workers & Model Services
Logs
Azure Alerting Baseline
Alerts
Troubleshooting Checklist

API Reference

`documentation/API_REFERENCE.md`

- Base URL
- Health
- Authentication (`/api/auth`)
- Organizations (`/api/orgs`)
- Job Roles (`/api/roles`)
  - Requirements Extraction (`/api/v1/roles`)
- Candidates (`/api/v1/candidates`)
  - Profile
  - Employment History
  - Education
  - Skills
  - Candidate-Facing Lists
  - Practice Interviews
  - Data Deletion (GDPR)
- Resume Batches (`/api/v1/resume-batches`)
- Applications (`/api/v1`)
- Invitations
- Interviews (`/api/v1/interviews`)
- AI Interview Chat (`/api/v1/interview`)
- Scoring (`/api/v1/scoring`)
- Interview Scores (`/api/v1/interview-scores`)
- Shortlist (`/api/v1/shortlist`)
- Storage (`/api/storage`)
- Azure Communication Services (`/api/v1/acs`)
- Call Automation (`/api/v1/call-automation`)
- Speech (`/api/v1/speech`)
- Data Retention (`/api/v1/data-retention`)
- Audit Log (`/api/v1/audit-log`)
- Notes

Database Schema

`documentation/DATABASE_SCHEMA.md`

- Overview
- Entity Relationship Diagram
- Tables
  - organisations
  - org_users
  - job_roles
  - candidate_profiles
  - candidate_skills
  - education
  - employment_history
  - candidate_dei
  - applications
  - interviews
  - transcript_segments
  - files
  - background_jobs
  - domain_events
  - interview_scores
  - score_dimensions
  - invitations
  - practice_interviews
  - user_roles
  - audit_log
  - data_deletion_requests
  - org_environment_inputs
  - resume_ingestion_batches
  - resume_ingestion_items
  - parsed_profile_snapshots
  - post_hire_outcomes
- Enums
  - app_role
  - interview_status
  - invitation_status
  - job_role_status
- Functions
  - user_belongs_to_org
  - user_org_role
  - get_user_org_id
  - has_role
- Access Control Policy Summary
- Data Flow Patterns
  - Candidate Application Flow

Data Retention Flow

Architecture Decisions

`documentation/ARCHITECTURE_DECISIONS.md`

ADR-001: FastAPI + PostgreSQL Backend
ADR-002: JWT Authentication
ADR-003: Azure Integrations
ADR-004: Frontend + API Contract
ADR-005: Database-Backed Background Jobs
ADR-006: Dual Scorecard Architecture
ADR-007: Decision-Dominant Scoring Logic
ADR-008: Python ACS Service Separation
ADR-009: Trunk-Based Release with Immutable Promotion

Frontend Architecture Guide

`documentation/FRONTEND_GUIDE.md`

Overview
Architecture Overview
- Technology Stack

Project Structure
Component Patterns
- Component File Structure
- Container vs Presentational Components
- shadcn/ui Component Customization

State Management
- React Query for Server State
- Local State Patterns
- State Flow Diagram

Routing
- Route Structure
- Protected Route Pattern
- Navigation Helpers

Form Handling
- React Hook Form + Zod Pattern
- Form with File Upload

Styling Conventions
- Design Token Usage
- Available Design Tokens
- Responsive Design
- Dark Mode Support

Hooks Library
- useCandidateData
- useOrgData
- useInterviewContext
- useAzureSpeech

API Integration
- FastAPI Client Usage
- AI Interview Calls

Performance Patterns
- Code Splitting
- Memoization
- Query Optimization
- Image Optimization

Testing (Recommended Patterns)
- Component Testing

Appendix: Component Hierarchy

User and Operational Workflows

`documentation/USER_GUIDE.md`

- Getting Started
  - Creating an Account
  - Accepting an Interview Invitation (Claim Flow)
  - Logging In
- For Candidates
  - Setting Up Your Profile
    - Personal Information
    - Uploading Your CV/Resume
    - Adding Skills
    - Work Experience
  - Practice Interviews
  - Taking a Real Interview
  - Managing Privacy
  - Requesting Data Deletion
- For Recruiters
  - Organisation Setup
  - Creating Job Roles
  - Customizing Scoring Rubric
  - Inviting Candidates
  - Bulk Resume Upload
  - Generating Shortlists
  - Reviewing Interviews
  - Exporting Reports
- For Administrators
  - Team Management
  - Organisation Settings
  - Recording Retention
  - Audit Logs
- Frequently Asked Questions
  - General
  - For Candidates
  - For Recruiters
- Troubleshooting
  - Camera/Microphone Not Working
  - Interview Won't Start
  - Invitation Link Expired
  - Audio Quality Issues
  - Video Freezing
  - Login Issues
- Contact Support
- Quick Reference Cards
  - Candidate Quick Start
  - Recruiter Quick Start

COMPILED SOURCE DOCUMENT

Platform Architecture Overview

`documentation/ARCHITECTURE_OVERVIEW.md`

Primary audit-ready architecture reference describing the system as built, including controls, deployment design, evidence, risks, and gap register items.

> **Version:** 2.0.0 **Date:** 19 April 2026 **Status:** Audit-ready — aligned to implementation evidence from repository **Supersedes:** Portions of HANDOVER.md, ARCHITECTURE_DIAGRAM.md, and ARCHITECTURE_DECISIONS.md (those documents remain as supporting references) **Author:** Architecture audit, April 2026

---

1. Document Purpose and Scope

This document provides a comprehensive, evidence-based description of the Talenti platform architecture as it exists in the repository at the time of writing. It is intended for use by:

- Engineering leadership and platform owners
- Security and compliance auditors
- Delivery and governance stakeholders
- New team members requiring architectural onboarding

Every significant claim in this document is traceable to repository evidence (file paths, configuration, code, infrastructure definitions, CI/CD workflows). Where evidence is incomplete, this is explicitly marked as a gap, assumption, or unresolved item.

Scope: This document covers the full Talenti platform including frontend, backend API, background worker, two ML model scoring services, an Azure Communication Services worker, infrastructure-as-code, CI/CD pipelines, data storage, authentication, security controls, and operational patterns.

Out of scope: Business requirements documentation, user research, commercial contracts, and Azure subscription-level governance (network security groups, Azure AD tenant configuration, subscription policies).

---

2. System Overview

Talenti is a B2B AI-powered interview platform that enables organisations to conduct, record, transcribe, score, and report on candidate interviews. The platform provides:

- A recruiter portal for managing job roles, candidates, applications, and interview results
- A candidate portal for profile management, interview preparation, and live AI-assisted interviews
- Automated dual-scorecard assessment combining culture/behavioural fit and skills/technical fit
- Azure-integrated video calling, recording, speech services, and AI interviewer capabilities
- Bulk resume ingestion with automated parsing and candidate invitation workflows

Service Inventory

Service	Technology	Port	Ingress	Repository Path
Frontend SPA	React 18 + Vite + Tailwind CSS	8080 (dev)	Public	`/src/`, `/public/`, `/index.html`
Backend API	FastAPI (Python 3.11)	8000	Public	`/backend/app/`
Backend Worker	Same codebase as Backend API	—	None (internal process)	`/backend/app/worker_main.py`
model-service-1	FastAPI (Python 3.11)	8001	Internal	`/model-service-1/`
model-service-2	FastAPI (Python 3.11)	8002	Internal	`/model-service-2/`
python-acs-service	FastAPI (Python 3.11)	8000 (internal)	Internal	`/python-acs-service/`
PostgreSQL	PostgreSQL 16	5432	Internal	Managed (Azure) / Container (local)

Evidence: Service definitions in `docker-compose.yml`, `infra/modules/platform.bicep`, individual `Dockerfile` files per service, `backend/app/main.py`, `model-service-1/app/main.py`, `model-service-2/app/main.py`, `python-acs-service/app/main.py`.

---

3. Business Purpose

Talenti is a hiring assessment platform designed for B2B use by recruiting teams and hiring managers. Its core proposition is:

1. **Structured AI interviews** — candidates interact with an AI interviewer powered by Azure OpenAI, with real-time speech and optional avatar rendering.
2. **Dual-scorecard assessment** — every interview produces two independent scores: culture/behavioural fit (5 canonical dimensions) and skills/technical fit (JD-driven competencies). These scores are never merged into a single composite.
3. **Recruiter decision support** — recruiters receive per-dimension scores, risk flags, confidence levels, and shortlist rankings to inform (not replace) hiring decisions.
4. **Compliance-oriented design** — deterministic scoring rules, audit logging, GDPR data deletion requests, and recording retention controls are built into the platform.

Evidence: Frontend pages (`src/lib/pages/`), backend API endpoints (`backend/app/api/`), model service scoring logic (`model-service-1/app/model.py`, `model-service-2/app/model.py`), database models for `audit_log`, `data_deletion_requests`, `recording_retention_days` on organisations.

---

4. Architectural Principles

The following principles are evident from the implementation:

Principle	Evidence
Separation of scoring concerns	Culture fit and skills fit are independent services with independent schemas, never merged (ADR-006)
Deterministic decision-making	ML extracts signals; deterministic rules with configurable thresholds produce all scoring decisions (ADR-007)
Immutable artifact promotion	Container images are built once on main, signed, attested, and promoted through environments without rebuilding (ADR-009, `ci-main.yml`, `promote-release.yml`)
Database-backed orchestration	Background jobs use a PostgreSQL table with polling rather than an external message broker (ADR-005, `background_jobs` model)
Azure-first infrastructure	All cloud services are Azure-native: Container Apps, PostgreSQL Flexible Server, Blob Storage, Communication Services, Speech, OpenAI, Key Vault, Front Door (Bicep templates in `infra/`)
Internal-only model and worker services	Only the backend API has public ingress; all other services are internal (`platform.bicep` ingress configuration)
Trunk-based development	All work merges to `main`; release-please manages versioning (ADR-009, `.release-please-manifest.json`)

---

5. System Context

External Actors

Actor	Interaction
Recruiters / Hiring Teams	Access recruiter portal via browser; manage roles, candidates, invitations, and interview results
Candidates	Access candidate portal via browser; manage profiles, join live interviews, complete practice sessions
Azure Communication Services	Provides identity tokens, video calling, call recording, and webhook events
Azure OpenAI	Powers the AI interviewer chat during live interviews
Azure Speech Services	Provides speech-to-text, text-to-speech, and avatar rendering tokens
Azure Blob Storage	Stores resumes, CVs, recordings, and upload artifacts
Azure Key Vault	Stores runtime secrets referenced by Container Apps
GitHub Actions	CI/CD platform for build, test, scan, sign, deploy, and promote workflows

System Boundary

The Talenti platform boundary encompasses:

- The React SPA (served from Azure Static Web Apps or Azure Storage + Front Door)
- The Backend API and Worker (Azure Container Apps)
- The two model services (Azure Container Apps, internal)
- The ACS worker service (Azure Container Apps, internal)
- The PostgreSQL database (Azure Database for PostgreSQL Flexible Server)
- Azure Blob Storage containers (`uploads` , `recordings`)

External to the boundary:

- Azure AD / identity provider for Azure subscription access (not used for application auth)
- Azure Communication Services control plane
- Azure OpenAI service endpoints
- Azure Speech service endpoints
- GitHub (source control and CI/CD)

---

6. High-Level Architecture

The system follows a microservices pattern with a central backend API as the control plane.

```
Recruiters / Candidates
        |
        v
  [Frontend SPA]  ──────→  [Azure Speech + ACS (browser-direct)]
        |
        v
  [Backend API]  ──────────→  [Azure OpenAI]
   |    |    |
   |    |    +──────────────→  [model-service-1: Culture Fit]
   |    |    +──────────────→  [model-service-2: Skills Fit]
   |    |    +──────────────→  [python-acs-service: Call Automation]
   |    |
   |    +──────────────────→  [Azure Blob Storage]
   |
   v
  [PostgreSQL]  ←──────────  [Backend Worker]
```

Key Architectural Boundaries

1. **Public ingress:** Only the frontend hosting and backend API are publicly accessible.
2. **Internal services:** Model services, ACS worker, and backend worker are internal only.
3. **Async orchestration:** The backend enqueues jobs in PostgreSQL; the worker claims and processes them.
4. **Dual scoring:** Culture fit and skills fit are structurally independent — different request schemas, different scoring engines, different output formats.

Evidence: `infra/modules/platform.bicep` (ingress configuration per Container App), `docker-compose.yml` (network topology), `backend/app/main.py` (router registration), `backend/app/services/ml_client.py` (HTTP client to model services).

---

7. Component Architecture

7.1 Frontend SPA

Attribute	Value
Framework	React 18, JavaScript (JSX, not TypeScript)
Build tool	Vite 6.0.5
Styling	Tailwind CSS 3.4.17 + shadcn/ui (Radix UI primitives)
State management	TanStack React Query 5 (server state), React Hook Form + Zod (forms)
Routing	React Router DOM 6.30.1
API client	Custom fetch-based HTTP client (`src/api/http.js`)
Key integrations	Azure Communication Services Calling SDK, Azure Speech SDK, jsPDF

Evidence: `package.json`, `src/lib/App.jsx`, `src/api/http.js`, `vite.config.js`, `tailwind.config.js`.

7.2 Backend API

Attribute	Value
Framework	FastAPI 0.111+
Language	Python 3.11
ORM	SQLAlchemy 2.0 with psycopg 3.2+
Migrations	Alembic 1.13+ (auto-run on startup)
Auth	JWT (HS256) via python-jose, bcrypt password hashing
Validation	Pydantic 2.8+
API routers	18 routers, ~91 endpoints

Evidence: `backend/pyproject.toml`, `backend/app/main.py`, `backend/app/core/config.py`, `backend/app/core/security.py`, `backend/app/api/`.

7.3 Backend Worker

Attribute	Value
Runtime	Same Python codebase as Backend API
Entry point	`python -m app.worker_main`
Pattern	Long-running async process polling `background_jobs` table
Job types	Resume parsing, auto-scoring, invitation prep, recording processing
Retry	Configurable max attempts with retry scheduling

Evidence: `backend/app/worker_main.py`, `backend/app/services/background_jobs.py`, `backend/app/services/job_handlers.py`, `docker-compose.yml` (backend-worker service).

7.4 Model Service 1 — Culture/Behavioural Fit

Attribute	Value
Framework	FastAPI
Model version	2.0.0
Engine	DecisionDominantEngine with KeywordSignalClassifier
Canonical dimensions	ownership, execution, challenge, ambiguity, feedback
Input	Interview transcript + operating environment + taxonomy
Output	Per-dimension scores, confidence, rationale, overall alignment, risk level, recommendation

Evidence: `model-service-1/app/main.py`, `model-service-1/app/model.py`, `model-service-1/app/schemas.py`, `model-service-1/requirements.txt`.

7.5 Model Service 2 — Skills/Technical Fit

Attribute	Value
Framework	FastAPI
Model version	3.0.0
Engine	DeterministicInterviewScorer (JD-driven)
Input	Job description + resume text + interview answers/transcript
Output	Overall score, per-competency scores (0-1), must-have pass/fail, gaps, outcome (PASS/REVIEW/FAIL)

Evidence: `model-service-2/app/main.py`, `model-service-2/app/model.py`, `model-service-2/app/schemas.py`, `model-service-2/requirements.txt`.

7.6 Python ACS Service

Attribute	Value
Framework	FastAPI
Purpose	Azure Communication Services call automation and recording
Auth	Shared secret (`ACS_WORKER_SHARED_SECRET`) for internal API
Endpoints	Call create/answer/hangup, recording start/pause/resume/stop, health checks
Dependencies	azure-communication-callautomation, azure-storage-blob, pydub (audio), ffmpeg

Evidence: `python-acs-service/app/main.py`, `python-acs-service/app/config.py`, `python-acs-service/app/api/routes/`, `python-acs-service/Dockerfile` (installs ffmpeg).

---

8. Frontend Architecture

8.1 Routing Structure

The SPA uses React Router v6 with the following route groups:

Route Pattern	Page Component	Purpose
`/`	Index	Landing page
`/auth`	Auth	Login and registration
`/org/*`	OrgDashboard, OrgOnboarding, OrgSettings, NewRole, RoleDetails, EditRoleRubric, InterviewReport	Recruiter portal
`/candidate/*`	CandidatePortal, CandidateProfile, PracticeInterview, InterviewLobby, LiveInterview, InterviewComplete	Candidate portal
`/invite/:token`	InviteValidation	Invitation claim flow

Evidence: `src/lib/App.jsx`.

8.2 API Client Pattern

The frontend uses a custom fetch-based HTTP client (`src/api/http.js`) with:

- Bearer token injection from `localStorage` (key: `talenti_auth_token`)
- Automatic Content-Type handling (JSON vs FormData)
- Custom `ApiError` class with status and error details
- Credentials: `include` for cookie-based refresh token support
- Snake_case/camelCase field mapping on specific API modules

Separate API modules exist for each domain: `auth.js` , `candidates.js` , `interviews.js` , `invitations.js` , `roles.js` , `organisations.js` , `scoring.js` , `resumeBatches.js` , `acs.js` , `speech.js` , `storage.js` , `audit.js` , `requirements.js` , `shortlist.js` .

Evidence: `src/api/http.js` , `src/api/auth.js` , and other files in `src/api/` .

8.3 State Management

- **Server state:** TanStack React Query 5 with domain-specific query keys (e.g., `["candidate-profile"]` , `["current-org"]` , `["job-roles", orgId]`).
- **Form state:** React Hook Form with Zod schema validation.
- **Local UI state:** React `useState` for toggles, loading indicators, and transient UI state.
- **No global auth context:** Auth state is derived from localStorage token and on-demand `authApi.me()` calls.

Custom hooks centralise data access: `useCandidateData.js` , `useOrgData.js` , `useInterviewPersistence.js` , `useAcsCall.js` , `useAzureSpeech.js` , `useAuditLog.js` .

Evidence: `src/lib/hooks/` , `package.json` (tanstack/react-query, react-hook-form, zod).

8.4 Browser-Direct Integrations

The frontend establishes direct connections to Azure services using backend-issued tokens:

- **Azure Communication Services Calling SDK** for video interview media
- **Azure Speech SDK** for speech-to-text and text-to-speech during interviews
- **Azure Speech Avatar** for visual avatar rendering

Evidence: `src/lib/hooks/useAcsCall.js`, `src/lib/hooks/useAzureSpeech.js`, `src/lib/hooks/useAzureAvatar.js`, `package.json` (@azure/communication-calling, microsoft-cognitiveservices-speech-sdk).

---

9. Backend Architecture

9.1 Application Structure

```
backend/
  app/
    main.py              # FastAPI app with lifespan, middleware, router registration
    worker_main.py       # Background worker entry point
    db.py                # SQLAlchemy engine and session factory
    core/
      config.py          # Pydantic Settings from environment
      security.py        # JWT creation, password hashing (bcrypt)
      logging.py         # JSON structured logging
      migrations.py      # Alembic auto-migration on startup
    api/
      deps.py            # Dependency injection (DB session, auth, org membership)
      auth.py            # Auth endpoints
      orgs.py            # Organisation endpoints
      roles.py           # Job role endpoints
      candidates.py      # Candidate profile endpoints
      applications.py    # Application endpoints
      interviews.py      # Interview lifecycle endpoints
      interview_scores.py
      scoring.py         # Dual-scorecard analysis endpoint
      resume_batches.py  # Bulk resume ingestion
      invitations.py     # Invitation management
      acs.py             # ACS token and webhook endpoints
      call_automation.py # Call control endpoints
      ai.py              # AI interviewer chat (Azure OpenAI)
      speech.py          # Speech token endpoint
      storage.py         # Blob storage SAS URL generation
      shortlist.py       # Candidate shortlisting
      retention.py       # Data retention cleanup
      audit_log.py       # Audit trail
      requirements.py    # JD parsing
    models/              # 27 SQLAlchemy ORM models
    schemas/             # Pydantic request/response schemas
    services/            # Business logic and external service clients
  alembic/               # Database migrations (8 versions)
  tests/                 # Pytest test suite
```

9.2 Middleware

The backend applies the following middleware (registered in `backend/app/main.py`):

1. **CORS Middleware** — Configurable origins from `ALLOWED_ORIGINS` environment variable; credentials enabled.
2. **Request Logging Middleware** — Generates `X-Request-ID` per request; logs method, path, status, and duration in structured JSON.

Evidence: `backend/app/main.py`.

9.3 Database Migrations

Alembic migrations run automatically on application startup via `run_startup_migrations()` in `backend/app/core/migrations.py`. The migration environment uses PostgreSQL advisory locking to prevent concurrent migration execution during multi-instance startups.

Migration history (8 versions):

1. `0001_initial.py` — Base schema
2. `0002_recording_lifecycle_fields.py` — Recording fields
3. `0003_blob_jobs_and_interview_lifecycle.py` — Blob jobs and interview management
4. `0004_resume_ingestion_and_claim_flow.py` — Resume parsing and invitation claim
5. `0005_scoring_canonical_dimensions.py` — Canonical scoring dimensions
6. `0006_org_environment_inputs.py` — Organisation environment configuration
7. `0007_post_hire_outcomes.py` — Post-hire outcome tracking
8. `0008_dual_scorecard_columns.py` — Dual scorecard support

Evidence: `backend/alembic/versions/`, `backend/alembic/env.py`, `backend/app/core/migrations.py`.

9.4 Key Runtime Flows

Interview Lifecycle:

1. `POST /api/v1/interviews/start` — Creates interview record, enqueues orchestration jobs
2. `POST /api/v1/interviews/{id}/transcripts` — Streams transcript segments during interview
3. `POST /api/v1/interviews/{id}/complete` — Marks interview complete, enqueues post-interview jobs
4. `POST /api/v1/scoring/analyze` — Backend calls model-service-1 and model-service-2, stores results in `interview_scores` and `score_dimensions`

Resume Batch Ingestion:

1. `POST /api/v1/resume-batches/` — Creates batch for a job role
2. `POST /api/v1/resume-batches/{id}/items/upload-url` — Generates SAS URL for direct-to-blob upload
3. `POST /api/v1/resume-batches/{id}/process` — Enqueues parsing jobs for all items
4. Backend worker parses resumes, creates parsed profile snapshots
5. `POST /api/v1/resume-batches/{id}/invite` — Sends invitations to parsed candidates

Invitation Claim Flow:

1. Candidate receives email with invitation link containing token
2. `GET /api/auth/claim-context?token=…` — Validates token, returns prefilled profile data
3. `POST /api/auth/claim-invite` — Claims account, sets up credentials, returns access token

---

10. Data Architecture

10.1 Database

Attribute	Value
Engine	PostgreSQL 16
ORM	SQLAlchemy 2.0 (declarative base)
Driver	psycopg 3.2+ (async-capable)
Migrations	Alembic with auto-run on startup
Local dev	Docker container (postgres:16-alpine) or external
Deployed	Azure Database for PostgreSQL Flexible Server (Burstable B1ms, 32GB storage, 7-day backup)

10.2 Core Entity Groups

Users and Organisations:

- `users` — Authentication accounts (email, password_hash, full_name, is_active)
- `organisations` — Tenant accounts with culture framework, retention settings
- `org_users` — Organisation membership with role (member/admin)
- `user_roles` — User role assignments

Candidates:

- `candidate_profiles` — Personal details, contact info, CV references, profile completion
- `employment_history`, `education`, `candidate_skills` — Profile components
- `candidate_dei` — Optional diversity/equity/inclusion data
- `parsed_profile_snapshots` — Resume parsing results

Recruitment:

- `job_roles` — Title, description, requirements, scoring rubric, interview structure
- `applications` — Links candidates to job roles with status tracking
- `invitations` — Token-based invitation with expiration and claim tracking
- `resume_ingestion_batches` / `resume_ingestion_items` — Bulk resume upload processing

Interviews and Scoring:

- `interviews` — Status, scheduling, call IDs, recording details, transcript status, anti-cheat signals
- `transcript_segments` — Speaker, content, timestamps per interview
- `interview_scores` — Scoring summary per interview
- `score_dimensions` — Per-dimension scores (culture and skills)
- `post_hire_outcomes` — Employment outcomes for future predictive analytics

Operations:

- `background_jobs` — Job queue (type, status, payload, attempts, retry scheduling)
- `domain_events` — Event outbox with JSON payloads
- `audit_log` — Action tracking for compliance
- `data_deletion_requests` — GDPR deletion request tracking
- `org_environment_inputs` — Culture/environment questionnaire responses

Evidence: `backend/app/models/`, `backend/alembic/versions/`, `documentation/DATABASE_SCHEMA.md`.

10.3 File Storage

Store	Purpose	Access Pattern
Azure Blob Storage (`uploads` container)	Resumes, CVs, profile documents	SAS URL upload from browser; backend/worker read
Azure Blob Storage (`recordings` container)	Interview call recordings	ACS worker writes; backend/worker reads
Local filesystem (fallback)	CV uploads when blob not configured	`POST /api/v1/candidates/cv` (local dev only)

Evidence: `infra/modules/platform.bicep` (storage containers), `backend/app/services/blob_storage.py`, `backend/app/core/config.py` (azure_storage_* settings), `python-acs-service/app/config.py` (RECORDING_CONTAINER).

10.4 Multi-Tenancy Model

The platform implements application-level multi-tenancy:

- Organisation isolation is enforced at the API layer via `require_org_member()` dependency injection
- Resources (job roles, applications, batches) are scoped to organisations via foreign keys
- There is no database-level row-level security (RLS) implemented despite documentation references

Note for auditors: The DATABASE_SCHEMA.md references RLS policies, but no RLS implementation was found in the Alembic migrations or model definitions. Multi-tenancy isolation relies entirely on application-layer checks in `backend/app/api/deps.py`.

Evidence: `backend/app/api/deps.py` (require_org_member), `backend/app/models/` (organisation_id foreign keys), `backend/alembic/versions/` (no RLS migration found).

---

11. Integration Architecture

11.1 Azure Service Integrations

Azure Service	Purpose	Integration Point	Credential Type
Azure Communication Services	Video calling, identity tokens, call recording	Backend API + ACS worker + browser SDK	Connection string
Azure OpenAI	AI interviewer chat responses	Backend API (`/api/v1/interview/chat`)	API key + endpoint
Azure Speech Services	Speech-to-text, text-to-speech, avatar	Backend API (token issuance) + browser SDK	API key + region
Azure Blob Storage	File uploads, recordings	Backend API (SAS URL generation), ACS worker, browser (direct upload)	Account key
Azure Key Vault	Runtime secret storage	Container Apps secret references	Managed identity (RBAC)
Azure Container Registry	Container image storage	CI/CD pipelines, Container Apps	Managed identity (AcrPull)
Azure Application Insights	Telemetry and diagnostics	Container Apps environment	Instrumentation key
Azure Log Analytics	Log aggregation and querying	Container Apps environment	Workspace ID
Azure Front Door	CDN and WAF for frontend (UAT/PROD)	Static website hosting	N/A

11.2 Inter-Service Communication

From	To	Protocol	Auth	Purpose
Backend API	model-service-1	HTTP POST `/predict`	None (internal network)	Culture fit scoring
Backend API	model-service-2	HTTP POST `/predict` or `/predict/transcript`	None (internal network)	Skills fit scoring
Backend API	python-acs-service	HTTP (various `/internal/*` endpoints)	Shared secret header	Call and recording management
Backend Worker	model-service-1	HTTP POST `/predict`	None (internal network)	Auto-scoring
Backend Worker	model-service-2	HTTP POST `/predict` or `/predict/transcript`	None (internal network)	Auto-scoring
Backend Worker	python-acs-service	HTTP	Shared secret header	Recording orchestration
python-acs-service	Backend API	HTTP callback (`/api/v1/acs/worker-events`)	Shared secret	Recording status updates
Azure ACS	Backend API	HTTP webhook (`/api/v1/acs/webhook`)	EventGrid subscription	Call lifecycle events
Browser	Azure ACS	WebRTC (via SDK)	ACS token (backend-issued)	Live call media
Browser	Azure Speech	WebSocket (via SDK)	Speech token (backend-issued)	Real-time speech

Note for auditors: Model services (model-service-1 and model-service-2) have no authentication on their `/predict` endpoints. They rely entirely on internal-only network ingress in the deployed Container Apps environment. In the local Docker Compose environment, these endpoints are accessible on localhost ports 8001 and 8002 without authentication.

Evidence: `backend/app/services/ml_client.py` , `backend/app/services/acs_worker_client.py` , `python-acs-service/app/security/internal.py` , `model-service-1/app/main.py` , `model-service-2/app/main.py` , `infra/modules/platform.bicep` .

---

12. Authentication and Authorization

12.1 Authentication Mechanism

Attribute	Value
Token type	JWT (JSON Web Tokens)
Algorithm	HS256 (symmetric)
Signing key	`JWT_SECRET` environment variable
Access token TTL	60 minutes (configurable)
Refresh token TTL	30 days (configurable)
Refresh token delivery	httpOnly cookie
Access token delivery	JSON response body, stored in `localStorage` on frontend
Password hashing	bcrypt via passlib
Token claims	`sub` (user ID), `iss`, `aud`, `exp`, optional extra claims

12.2 Authentication Flows

1. **Registration:** `POST /api/auth/register` — Creates user, returns access token
2. **Login:** `POST /api/auth/login` — Validates credentials, returns access token, sets refresh cookie
3. **Token refresh:** `POST /api/auth/refresh` — Uses refresh token cookie, returns new access token
4. **Invitation claim:** `POST /api/auth/claim-invite` — Claims invitation token, creates/updates user, returns access token
5. **Session check:** `GET /api/auth/me` — Returns current user from JWT

12.3 Authorization Model

- **JWT validation:** Every authenticated endpoint uses `get_current_user()` dependency which decodes and validates the JWT, then loads the user from the database.
- **Organisation membership:** Endpoints requiring org context use `require_org_member()` which checks the `org_users` table.
- **Role-based access:** The `org_users.role` field supports `member` and `admin` roles, though the enforcement granularity of admin-vs-member permissions at the endpoint level is limited in evidence.
- **No RBAC framework:** There is no explicit role-based access control middleware or permission decorator system. Authorization checks are implemented inline in route handlers.

Evidence: `backend/app/core/security.py`, `backend/app/api/deps.py`, `backend/app/api/auth.py`.

12.4 Internal Service Authentication

The python-acs-service uses a shared secret (`ACS_WORKER_SHARED_SECRET`) for authenticating requests from the backend. This is validated in `python-acs-service/app/security/internal.py`.

Model services have no authentication — they rely on network-level isolation (internal-only ingress in Container Apps).

---

13. Security Architecture and Controls

13.1 Implemented Controls

Control	Implementation	Evidence
JWT authentication	HS256 tokens with issuer/audience validation	`backend/app/core/security.py`, `backend/app/api/deps.py`
Password hashing	bcrypt via passlib	`backend/app/core/security.py`
CORS	Configurable allowed origins	`backend/app/main.py`, `backend/app/core/config.py`
HTTPS	Enforced at infrastructure level (Front Door, App Service)	`infra/modules/platform.bicep`
Secret scanning	Gitleaks in PR pipeline	`.github/workflows/pr-security-iac.yml`
Dependency scanning	npm audit, pip-audit, Trivy	`.github/workflows/pr-security-iac.yml`, `.github/workflows/ci-main.yml`
Container scanning	Trivy vulnerability scan (high/critical gate)	`.github/workflows/ci-main.yml`
IaC scanning	Checkov policy scan on Bicep templates	`.github/workflows/pr-security-iac.yml`
CodeQL	Static analysis for JS/TS, Python, Actions	`.github/workflows/codeql.yml`
Image signing	Cosign keyless signing via OIDC	`.github/workflows/ci-main.yml`
Build attestation	GitHub build provenance attestation	`.github/workflows/ci-main.yml`
SBOM generation	Syft (SPDX format) for built images	`.github/workflows/ci-main.yml`
Dockerfile linting	Hadolint	`.github/workflows/pr-security-iac.yml`
WAF	Azure Front Door WAF policy (UAT/PROD)	`infra/uat/main.bicep`, `infra/prod/main.bicep`
Audit logging	`audit_log` table with action tracking	`backend/app/api/audit_log.py`, `backend/app/models/`
GDPR deletion	`data_deletion_requests` table and candidate deletion endpoint	`backend/app/api/candidates.py`
Recording retention	Configurable per-organisation retention period	`organisations.recording_retention_days`, `backend/app/api/retention.py`
Non-root containers	All Dockerfiles create and run as non-root users	`backend/Dockerfile`, `model-service-1/Dockerfile`, etc.
Internal-only services	Model services and ACS worker have internal-only ingress	`infra/modules/platform.bicep`

Control	Implementation	Evidence
Shared secret auth	ACS worker endpoints require shared secret	`python-acs-service/app/security/internal.py`
SAS URL expiry	Blob storage SAS tokens expire after configurable TTL (default 15 min)	`backend/app/core/config.py`

13.2 Security Considerations and Gaps

Item	Status	Notes
Access token in localStorage	**Risk**	Vulnerable to XSS; httpOnly cookie would be more secure for the access token
HS256 JWT algorithm	**Acceptable for single-service**	Symmetric key shared by backend only; asymmetric (RS256) would be needed for multi-party verification
Model service no-auth	**Mitigated by network**	Relies entirely on Container Apps internal ingress; no defence-in-depth
No rate limiting	**Gap**	No evidence of rate limiting on authentication or API endpoints
No CSRF protection	**Partial**	Refresh token in httpOnly cookie but no CSRF token mechanism observed
No IP allowlisting on backend	**Gap**	Backend API is publicly accessible; only frontend has WAF/Front Door in UAT/PROD
RLS not implemented	**Gap**	Documentation references RLS but no implementation found; relies on application-layer isolation
`datetime.utcnow()` usage	**Minor**	Deprecated in Python 3.12+; should use `datetime.now(UTC)`

Evidence: Source code files listed above, CI/CD workflow files, infrastructure templates.

---

14. Configuration and Secrets Management

14.1 Configuration Approach

The backend uses Pydantic `BaseSettings` to load configuration from environment variables with `.env` file fallback. The settings class is defined in `backend/app/core/config.py`.

14.2 Key Environment Variables

Variable	Service	Purpose	Sensitivity
`DATABASE_URL`	Backend, ACS worker	PostgreSQL connection string	High (contains credentials)
`JWT_SECRET`	Backend	JWT signing key	Critical
`AZURE_STORAGE_ACCOUNT_KEY`	Backend	Blob storage access	High
`AZURE_ACS_CONNECTION_STRING`	Backend, ACS worker	Communication Services	High
`AZURE_OPENAI_API_KEY`	Backend	AI service access	High
`AZURE_SPEECH_KEY`	Backend	Speech service access	High
`ACS_WORKER_SHARED_SECRET`	Backend, ACS worker	Inter-service auth	High
`MODEL_SERVICE_1_URL`	Backend	Model service endpoint	Low
`MODEL_SERVICE_2_URL`	Backend	Model service endpoint	Low
`ALLOWED_ORIGINS`	Backend, ACS worker	CORS configuration	Low
`AUTO_SCORE_INTERVIEWS`	Backend worker	Feature flag	Low
`BACKGROUND_WORKER_POLL_INTERVAL_SECONDS`	Backend worker	Worker tuning	Low

14.3 Secrets in Deployed Environments

In Azure Container Apps, secrets are referenced from Azure Key Vault via managed identity with `Key Vault Secrets User` RBAC role assignment. This is configured in the infrastructure deployment workflows.

Evidence: `backend/app/core/config.py`, `.env.example`, `infra/modules/platform.bicep` (Key Vault resource), `.github/workflows/infra-dev.yml` (role assignments).

14.4 Local Development

Local development uses `.env` files at the repository root and in `/backend/`. The `.env.example` template provides the required variable list. Default `DATABASE_URL` points to `localhost:5432`.

---

15. Environments and Deployment Architecture

15.1 Environment Matrix

Environment	Frontend Hosting	Backend Hosting	Database	Ingress	Network
Local	Vite dev server (8080)	Uvicorn (8000)	Docker PostgreSQL	localhost	Docker bridge network
DEV	Azure Static Web Apps	Azure Container Apps	Azure PostgreSQL Flexible Server	Public (backend), internal (services)	Container Apps environment
UAT	Azure Storage + Front Door (WAF + IP allowlist)	Azure Container Apps	Azure PostgreSQL Flexible Server	Public (backend), internal (services)	Container Apps environment
PROD	Azure Storage + Front Door (WAF)	Azure Container Apps	Azure PostgreSQL Flexible Server	Public (backend), internal (services)	Container Apps environment

15.2 Azure Resource Inventory per Environment

Each environment deploys the following Azure resources (defined in `infra/modules/platform.bicep`):

Resource	SKU/Tier
Container Apps Environment	Managed (with Log Analytics)
Container Apps (5)	0.5 CPU / 1Gi memory per container, scale 1-2 replicas
Azure Container Registry	Basic
Azure Key Vault	Standard (RBAC enabled)
Azure Storage Account	Standard_LRS, StorageV2, TLS 1.2
Azure Database for PostgreSQL Flexible Server	Burstable B1ms, 32GB storage, 7-day backup
Log Analytics Workspace	PerGB2018, 30-day retention
Application Insights	Web type
Azure Monitor Action Group	Email alerts

Additionally for UAT/PROD:

Resource	Details
Azure Front Door Standard	With WAF policy
Storage static website hosting	Enabled via workflow

15.3 Container App Configuration

Container App	Image Source	Ingress	Replicas	Health Probes
backend	ACR (SHA-tagged)	External, port 8000	1-2	Liveness + Readiness on `/health`
backend-worker	Same image as backend	None	1	Liveness + Readiness
model-service-1	ACR (pinned digest)	Internal, port 8001	1-2	Liveness + Readiness on `/health`
model-service-2	ACR (pinned digest)	Internal, port 8002	1-2	Liveness + Readiness on `/health`
acs-worker	ACR (SHA-tagged)	Internal, port 8000	1	Liveness + Readiness

Evidence: `infra/modules/platform.bicep`, `infra/dev/parameters.dev.json`, `infra/uat/parameters.uat.json`, `infra/prod/parameters.prod.json`.

---

16. CI/CD and Release Approach

16.1 Pipeline Overview

The platform uses GitHub Actions with 11 workflow files implementing a trunk-based development model with immutable artifact promotion.

```
Feature PR
  ├── pr-fast-quality (lint, test, build, coverage gates)
  ├── pr-security-iac (secrets, dependencies, containers, IaC)
  ├── pr-ephemeral-deploy (isolated PR environment + smoke tests)
  ├── codeql (static analysis)
  v
Merge to main
  ├── ci-main (build, scan, sign, attest images)
  ├── infra-dev (validate + deploy infrastructure if changed)
  v
deploy-dev (deploy to DEV, smoke test)
  v
release (release-please versioning, manifest creation)
  v
promote-release → UAT (automatic on release publish)
  v
promote-release → PROD (manual workflow_dispatch by tag)
```

16.2 PR Quality Gates

Workflow	Checks
`pr-fast-quality.yml`	Conventional Commit title format, frontend lint + test + build, backend tests (60% coverage gate), ACS service tests (40% coverage gate), migration execution check
`pr-security-iac.yml`	Gitleaks secret scanning, npm audit (high/critical), pip-audit (strict), Dockerfile linting (hadolint), Trivy container scan, Bicep compilation, Checkov IaC policy scan
`pr-ephemeral-deploy.yml`	Builds PR-specific image, creates isolated database and Container App, runs migrations, smoke tests, auto-cleanup on PR close
`codeql.yml`	GitHub CodeQL analysis for JavaScript/TypeScript, Python, and Actions

16.3 Immutable Image Pipeline

1. `ci-main.yml` builds backend and ACS worker images tagged with the Git SHA
2. Generates SBOMs using Syft (SPDX format)
3. Scans images with Trivy (high/critical vulnerability gate)
4. Signs images with cosign (keyless OIDC)
5. Attests build provenance via GitHub `actions/attest-build-provenance`
6. Uploads CI evidence artifacts (manifests, SBOMs, signatures)

16.4 Release and Promotion

- **release-please** manages `VERSION` file and `CHANGELOG.md` via conventional commit parsing
- `release-manifest.json` is the promotion contract: contains version, git SHA, all image digests, frontend source SHA
- UAT auto-promotes from published GitHub Releases
- PROD requires manual `workflow_dispatch` specifying a release tag
- `promote-release.yml` imports images into target ACR, verifies signatures with cosign, deploys without rebuilding

16.5 Infrastructure as Code

- Bicep templates in `infra/modules/platform.bicep` (shared module) with per-environment parameter files
- Separate infra workflows per environment (`infra-dev.yml` , `infra-uat.yml` , `infra-prod.yml`)
- `deploy-dev.yml` gates on `infra-dev` success when infrastructure files change
- OIDC federation for Azure authentication in workflows

Evidence: All files in `.github/workflows/` , `infra/` , `.release-please-manifest.json` , `VERSION` , `CHANGELOG.md` .

---

17. Logging, Monitoring, and Operational Support

17.1 Logging

Aspect	Implementation
Format	Structured JSON to stdout
Request tracing	`X-Request-ID` generated per request, included in all log entries
Worker logging	`correlation_id`, `job_type`, `job_id` context variables
Log aggregation	Azure Log Analytics via Container Apps environment
Framework	Python `logging` with custom `JsonFormatter`

Evidence: `backend/app/core/logging.py`, `backend/app/main.py` (middleware), `python-acs-service/app/logging_utils.py`.

17.2 Monitoring

Component	Health Endpoint	Monitoring
Backend API	`GET /health`	Application Insights, synthetic availability
model-service-1	`GET /health`	Container restart count, prediction latency
model-service-2	`GET /health`	Container restart count, prediction latency
python-acs-service	`GET /health`, `/health/ready`, `/health/live`	Readiness checks (ACS, storage, callback connectivity)
Backend Worker	N/A (no HTTP endpoint)	Queue metrics: `pending_jobs`, `oldest_pending_job_age_seconds`
PostgreSQL	`pg_isready`	Azure managed server health, disk usage

17.3 Alerting Baseline

The infrastructure provisions Azure Monitor resources per environment:

- Log Analytics workspace
- Application Insights component
- Azure Monitor Action Group (email-based)

Documented alert targets (configuration details not found in IaC):

- Synthetic availability checks for frontend and backend
- 5xx spike metric alerts
- Latency regression alerts
- Container restart count alerts
- Background job backlog and age query alerts

Note for auditors: Alert rule definitions are documented in `MONITORING.md` but the actual Azure Monitor alert rules are not defined in the Bicep templates. Alert configuration may exist as manual Azure portal configuration or in a separate operational runbook.

Evidence: `infra/modules/platform.bicep` (Log Analytics, App Insights, Action Group resources), `documentation/MONITORING.md`.

18. Error Handling and Resilience Considerations

18.1 Implemented Patterns

Pattern	Implementation
Database connection retry	Alembic advisory locking prevents concurrent migration failures
Background job retry	Configurable max attempts with retry scheduling in `background_jobs`
Health checks	All services expose health endpoints; Container Apps use liveness/readiness probes
Graceful degradation	Blob storage falls back to local file system when not configured
Domain event outbox	`domain_events` table ensures events survive worker restarts

18.2 Resilience Gaps

Gap	Risk
No circuit breaker for external Azure service calls	Azure service outages could cascade to all endpoints
No request timeout configuration on inter-service HTTP calls	Slow model service responses could exhaust backend connections
Single-instance backend worker	Worker failure stops all async processing until restart
No dead-letter queue	Failed jobs after max retries remain in `failed` status but have no automated alerting or reprocessing
No database connection pooling configuration	Default SQLAlchemy pool settings may not be optimal for production concurrency

19. Architecture Decision Records

ADR-001: FastAPI + PostgreSQL Backend

Field	Value
Status	Accepted
Context	The platform needed a lightweight, Python-based web framework with strong async support and a reliable relational database.
Decision	Use FastAPI for the backend API layer and PostgreSQL as the primary datastore.
Rationale	Simple local development experience. Consistent with Python ML ecosystem for model services. PostgreSQL provides ACID transactions for job queue and audit trail.
Implications	All services use Python 3.11. Single database for all backend state. No separate caching layer.
Evidence	`backend/pyproject.toml`, `backend/app/main.py`, `docker-compose.yml`, `infra/modules/platform.bicep`.

ADR-002: JWT Authentication with Local User Table

Field	Value
Status	Accepted
Context	The platform needed authentication that works in local development without third-party providers.
Decision	Use JWTs issued and validated by the FastAPI service with a local `users` table. HS256 algorithm with configurable TTL.
Rationale	Keeps auth within the service boundary. Enables standalone operation. Refresh tokens in httpOnly cookies provide session continuity.
Implications	No SSO/SAML/OIDC support for enterprise customers. JWT secret must be securely managed. No token revocation mechanism beyond expiry.
Evidence	`backend/app/core/security.py`, `backend/app/api/auth.py`, `backend/app/api/deps.py`.

ADR-003: Azure-First Cloud Services

Field	Value
Status	Accepted
Context	The platform requires video calling, recording, speech services, AI capabilities, and object storage.
Decision	Use Azure Communication Services, Azure Speech, Azure OpenAI, and Azure Blob Storage as managed services.
Rationale	Provides scalable managed services with consistent SDK experience. Aligns with organisational cloud strategy.
Implications	Strong Azure lock-in. All credentials managed via environment variables, with Key Vault in production.
Evidence	`backend/app/core/config.py` (Azure config variables), `infra/modules/platform.bicep`, `python-acs-service/`, `src/lib/hooks/useAcsCall.js`.

ADR-004: REST API with Versioned Prefixes

Field	Value
Status	Accepted
Context	The frontend needs a stable API contract with the backend.
Decision	Use REST endpoints under `/api` and `/api/v1` prefixes.
Rationale	Clear versioning enables API evolution. Compatible with frontend fetch-based client.
Implications	Some endpoints use `/api` (auth, orgs, invitations) and some use `/api/v1` (candidates, interviews, scoring). This split is historical rather than intentional versioning.
Evidence	`backend/app/main.py` (router prefix registration).

ADR-005: Database-Backed Background Jobs

Field	Value
Status	Accepted
Context	Async operations (resume parsing, scoring, invitation prep, recording processing) need reliable execution without introducing an external message broker.
Decision	Use a `background_jobs` table polled by `backend-worker` instead of RabbitMQ, Azure Service Bus, or similar.
Rationale	Avoids external broker dependency. Jobs are transactionally consistent with the state changes that produce them. The `domain_events` outbox ensures events are not lost. Retry logic with exponential backoff is built into the worker.
Implications	Polling introduces latency (configurable, default 2s). Single-instance worker is a scaling constraint. No native dead-letter or fan-out patterns.
Evidence	`backend/app/models/` (BackgroundJob model), `backend/app/worker_main.py`, `backend/app/services/background_jobs.py`, `docker-compose.yml` (backend-worker service).

ADR-006: Dual Scorecard Architecture

Field	Value
Status	Accepted
Context	Interview assessment needs to evaluate both culture fit and skills fit, but these are fundamentally different types of assessment.
Decision	Split scoring into two independent model services. model-service-1 handles culture/behavioural fit (5 canonical dimensions). model-service-2 handles skills/technical fit (JD-driven competencies). Scores are never merged into a single composite.
Rationale	Culture fit and skills fit are orthogonal assessments. Independent services can evolve, retrain, or be replaced independently. Recruiters can weight each dimension on its own terms.
Implications	Frontend must display two independent scorecards. Backend orchestrates calls to both services. Interview reports show dual perspectives.
Evidence	`model-service-1/app/schemas.py`, `model-service-2/app/schemas.py`, `backend/app/api/scoring.py`, `backend/alembic/versions/0008_dual_scorecard_columns.py`.

ADR-007: Decision-Dominant Scoring Logic

Field	Value
Status	Accepted
Context	Hiring decisions based on AI scoring must be auditable and legally defensible.
Decision	ML classifiers extract signals from interview transcripts, but all scoring decisions are made by deterministic rules with configurable thresholds.
Rationale	Deterministic decisions are auditable and explainable. Scoring rules can be described to candidates and recruiters. Confidence gating flags uncertainty. Fatal risk flags override all other signals.
Implications	Scoring accuracy is bounded by rule quality, not model capability. Rules must be maintained and version-controlled alongside the model.
Evidence	`model-service-1/app/model.py` (DecisionDominantEngine), `model-service-2/app/model_draft.py` (DeterministicInterviewScorer).

ADR-008: Python ACS Service Separation

Field	Value
Status	Accepted
Context	Azure Communication Services call automation has different scaling characteristics and failure modes from the main API.
Decision	Run ACS call automation as a separate microservice (`python-acs-service`) with internal-only ingress.
Rationale	Isolates call-related failures from core API. Enables independent scaling. Reduces attack surface via internal-only ingress.
Implications	Requires shared-secret authentication for inter-service calls. Adds deployment complexity (separate image, separate Container App).
Evidence	`python-acs-service/` , `infra/modules/platform.bicep` (internal ingress), `docker-compose.yml` (acs-worker service).

ADR-009: Trunk-Based Release with Immutable Promotion

Field	Value
Status	Accepted
Context	The platform needs a reliable release process that eliminates environment-specific build differences.
Decision	Use trunk-based development on `main` with release-please for versioning. Container images are built once, signed, attested, and promoted immutably through DEV -> UAT -> PROD.
Rationale	Build-once-promote-many eliminates environment-specific build issues. Image signing and attestation provide supply-chain integrity. `release-manifest.json` captures exact digests for reproducible deployments.
Implications	All environment-specific configuration must be external (env vars, Key Vault). Model service images are pinned by digest, not built in this pipeline. Frontend is rebuilt per environment from source (not promoted as an image).
Evidence	`.github/workflows/ci-main.yml`, `.github/workflows/deploy-dev.yml`, `.github/workflows/release.yml`, `.github/workflows/promote-release.yml`, `.release-please-manifest.json`.

---

20. Known Limitations, Technical Debt, and Risks

20.1 Technical Debt

Item	Description	Severity
Deprecated API client files	`src/lib/lib/apiClient.js` and `src/lib/lib/authClient.js` appear to be legacy predecessors of the current `src/api/` modules	Low
Mixed API prefix convention	Some endpoints use `/api` and others use `/api/v1` without clear versioning strategy	Low
`datetime.utcnow()` usage	Used in JWT creation; deprecated in Python 3.12+	Low
Frontend TypeScript absence	Project uses JavaScript (JSX) without TypeScript, relying on Zod for runtime validation only	Medium
No OpenAPI documentation	FastAPI auto-generates OpenAPI docs, but no evidence of custom documentation or schema validation tests	Low

20.2 Architectural Risks

Risk	Impact	Likelihood	Mitigation
Single-instance worker	Async job processing stops if worker crashes	Medium	High
No rate limiting	Authentication and API endpoints vulnerable to brute force	High	Medium
Access token in localStorage	XSS vulnerability could expose tokens	High	Low-Medium
No token revocation	Compromised tokens valid until expiry (60 min)	Medium	Low
Azure service dependency	Multiple critical paths depend on Azure availability	High	Low
Model service no-auth	Compromised network could allow unauthorized scoring	Medium	Low

20.3 Known Limitations

- No offline or degraded-mode operation — the platform requires all Azure services to be available
- No horizontal scaling of the background worker beyond a single instance
- No WebSocket or server-sent events — the frontend polls for updates
- No internationalisation (i18n) support
- No accessibility (a11y) audit evidence

---

21. Assumptions and Unresolved Questions

Assumptions

1. **Azure AD tenant governance** is managed outside this repository at the subscription level.
2. **Network security groups and VNet peering** for UAT/PROD are configured outside the Bicep templates (the `platform.bicep` module references VNet parameters but does not create network security rules).
3. **Model service images** are built and published in separate repositories and pinned by digest in the deployment environment variables.
4. **Alert rule configuration** exists in Azure portal or a separate operational repository, as the Bicep templates provision monitoring resources but not alert rules.
5. **SSL/TLS certificates** are managed by Azure (Static Web Apps, Front Door, Container Apps provide automatic TLS).

Unresolved Questions

1. **RLS implementation status:** DATABASE_SCHEMA.md documents Row-Level Security policies, but no RLS implementation was found in migrations. Is this planned or abandoned?
2. **Admin vs Member permission enforcement:** The `org_users.role` field supports roles, but how granularly are admin-only operations enforced at the endpoint level?
3. **Model service repository locations:** The integration guide references external git repositories for model services, but the actual repository URLs are not documented.
4. **Data retention automation:** The `/api/v1/retention/cleanup` endpoint exists, but is retention enforcement automated (scheduled job) or manual?
5. **Post-hire outcome data collection:** The `post_hire_outcomes` table exists but the ingestion mechanism (API endpoint, manual import, integration) is not clear.
6. **Practice interview scoring:** Practice interviews are stored, but whether they use the same model scoring pipeline or a separate mechanism is unclear.

Appendix A: Repository Evidence Summary

Directory Structure

```
/
  .github/workflows/     # 11 CI/CD workflow files
  backend/
    app/
      api/               # 18 API router modules + deps.py
      core/              # config, security, logging, migrations
      models/            # 27 SQLAlchemy ORM models
      schemas/           # Pydantic request/response schemas
      services/          # Business logic and external service clients
    alembic/versions/    # 8 migration files
    tests/               # 8 test modules
  model-service-1/
    app/                 # FastAPI app, model, schemas, utils
    models/              # Decision engine and classifier
    prompts/             # Prompt templates
    schemas/             # Taxonomy definitions
    tests/               # 4 test modules
  model-service-2/
    app/                 # FastAPI app, model, schemas
    models/              # Scorer and JD parser
    tests/               # 6 test modules
  python-acs-service/
    app/                 # FastAPI app, routes, config, security
    alembic/             # Service-specific migrations
    tests/               # 4 test modules
  src/
    api/                 # 14 API client modules
    lib/
      components/        # UI and domain components
      hooks/             # 16 custom React hooks
      pages/             # 19 page components
      lib/               # Utility modules
  infra/
    modules/             # platform.bicep (shared module)
    dev/                 # Dev environment Bicep + parameters
    uat/                 # UAT environment Bicep + parameters
    prod/                # Prod environment Bicep + parameters
  documentation/         # 12 markdown documentation files
  scripts/               # Automation and setup scripts
  tools/                 # Additional tooling
```

Key Configuration Files

File	Purpose
`package.json`	Frontend dependencies and scripts
`backend/pyproject.toml`	Backend Python dependencies
`model-service-1/requirements.txt`	Culture fit service dependencies
`model-service-2/requirements.txt`	Skills fit service dependencies
`python-acs-service/requirements.txt`	ACS service dependencies
`docker-compose.yml`	Local development orchestration
`.env.example`	Environment variable template
`infra/modules/platform.bicep`	Shared infrastructure module
`.release-please-manifest.json`	Release versioning manifest
`VERSION`	Current version (0.2.0)

Test Coverage

Service	Test Directory	Test Files	Coverage Gate
Backend	`backend/tests/`	8 modules	60% (CI enforced)
model-service-1	`model-service-1/tests/`	4 modules	Not enforced in CI
model-service-2	`model-service-2/tests/`	6 modules	Not enforced in CI
python-acs-service	`python-acs-service/tests/`	4 modules	40% (CI enforced)
Frontend	N/A	Referenced in CI but location unclear	Referenced but not enforced

---

Appendix B: Audit Findings — Contradictions and Stale Documentation

B.1 Contradictions Between Documentation and Implementation

#	Document	Claim	Reality	Severity
1	`DATABASE_SCHEMA.md`	References RLS (Row-Level Security) policies for multi-tenancy	No RLS implementation found in any Alembic migration or model definition. Multi-tenancy relies on application-layer checks in `deps.py`.	High — misleading for auditors
2	`DATABASE_SCHEMA.md`	References `auth_users` table in ER diagram context	The actual table is `users` based on model definitions	Medium — naming inconsistency
3	`FRONTEND_GUIDE.md` (v1.1.0)	References `.tsx` file extensions in some places	All frontend files use `.jsx` (JavaScript, not TypeScript)	Low — cosmetic
4	`DEPLOYMENT_DEV_V2.md`	States "all core DEV infra remains in australiaeast"	DEV frontend uses Azure Static Web Apps which may be in `eastasia` (closest available region)	Low — region ambiguity for non-critical service
5	`API_REFERENCE.md`	Documents `POST /api/storage/upload-url`	Actual route is `POST /api/v1/storage/upload-url` based on router registration	Low — prefix mismatch

B.2 Stale or Incomplete Documentation

#	Document	Issue
1	`AZURE_DEV_UAT_PROD_SETUP.txt`	Plain text file with manual Azure portal setup notes. Superseded by Bicep IaC and GitHub Actions workflows. Should be marked as legacy.
2	`CODEX_CAPABILITY_SETUP.md`	References GitHub Codex/Copilot setup and OIDC bootstrapping. Operational setup doc, not architecture. Content validity not verified.
3	`INTEGRATION_GUIDE.md`	References git submodule approach for model services, but current deployment uses pinned ACR image digests. The integration approach has evolved.
4	`old/` directory	Contains archived documents (FRONTEND_MIGRATION_PLAN.md, MODEL_SERVICE_SETUP_PROMPT.md). Correctly moved to old/ but their existence should be noted.
5	`model-service-1/IMPLEMENTATION_CHECKLIST.md`, `SETUP_COMPLETE.md`	Setup-phase documents that are no longer relevant post-implementation
6	Various `PROMPT_*.txt` and `PROMPT_*.md` files at root	AI prompt engineering documents used during development; not architecture documentation

B.3 Documentation Gaps

#	Gap	Impact
1	No single canonical architecture document existed before this audit	Auditors would need to piece together architecture from 12+ documents
2	No data flow diagram showing PII handling	Required for privacy/GDPR compliance review
3	No threat model or security architecture diagram	Required for security audit
4	No disaster recovery or business continuity documentation	Required for operational risk assessment
5	No capacity planning or performance baseline documentation	Required for scalability review
6	No API authentication/authorization matrix	Required for access control review
7	Alert rule definitions not in IaC	Monitoring configuration not reproducible from repository

---

Appendix C: Gap Register for Auditors

C.1 Documentation Gaps

ID	Gap	Priority	Recommendation
DOC-01	No canonical architecture document	**Resolved**	This document serves as the canonical reference
DOC-02	No PII data flow diagram	High	Create a data flow diagram showing PII collection, storage, processing, and retention paths
DOC-03	No threat model	High	Conduct threat modelling (STRIDE or equivalent) for the platform
DOC-04	No disaster recovery documentation	Medium	Document RPO/RTO targets, backup/restore procedures, and failover processes
DOC-05	No API authorization matrix	Medium	Create a matrix mapping endpoints to required roles/permissions
DOC-06	DATABASE_SCHEMA.md claims RLS that does not exist	High	Remove RLS references or implement RLS
DOC-07	AZURE_DEV_UAT_PROD_SETUP.txt is stale	Low	Mark as superseded by IaC
DOC-08	INTEGRATION_GUIDE.md partially stale	Low	Update to reflect current ACR digest-based integration

C.2 Implementation Gaps

ID	Gap	Priority	Recommendation
IMP-01	No rate limiting on API endpoints	High	Implement rate limiting on auth endpoints at minimum
IMP-02	No CSRF protection for cookie-based refresh	Medium	Add CSRF token for state-changing requests that use cookies
IMP-03	RLS not implemented despite documentation	Medium	Implement database-level RLS or explicitly document application-layer-only isolation as a design decision
IMP-04	Model services have no authentication	Medium	Add shared-secret or mTLS authentication as defence-in-depth
IMP-05	No circuit breaker for external service calls	Medium	Implement circuit breaker pattern for Azure service integrations
IMP-06	No request timeout on inter-service HTTP calls	Medium	Configure explicit timeouts on `ml_client.py` and `acs_worker_client.py`
IMP-07	Alert rules not defined in IaC	Medium	Add Azure Monitor alert rules to Bicep templates
IMP-08	Frontend test coverage not enforced in CI	Low	Add frontend test coverage gate to `pr-fast-quality.yml`

C.3 Security/Control Gaps

ID	Gap	Priority	Recommendation
SEC-01	Access token stored in localStorage (XSS risk)	Medium	Evaluate migration to httpOnly cookie for access token
SEC-02	No token revocation mechanism	Medium	Implement token blocklist or reduce access token TTL
SEC-03	HS256 JWT algorithm (symmetric key)	Low	Acceptable for single-service, but document the decision. Consider RS256 if multi-party token verification is needed.
SEC-04	No penetration test evidence	High	Conduct and document penetration testing
SEC-05	No DAST (Dynamic Application Security Testing) in pipeline	Medium	Add DAST scanning to CI/CD
SEC-06	Backend API publicly accessible without WAF	Medium	Route backend traffic through Front Door WAF in UAT/PROD
SEC-07	`datetime.utcnow()` deprecated	Low	Update to `datetime.now(datetime.UTC)` for Python 3.12+ compatibility

C.4 Operational Gaps

ID	Gap	Priority	Recommendation
OPS-01	Single-instance backend worker	Medium	Document scaling strategy or implement multi-instance job claiming
OPS-02	No dead-letter handling for failed jobs	Medium	Implement alerting and reprocessing for permanently failed jobs
OPS-03	No documented incident response procedure	Medium	Create runbook for common failure scenarios
OPS-04	No backup/restore testing evidence	Medium	Document and test database restore procedures
OPS-05	No capacity planning baseline	Low	Establish performance baselines and scaling triggers
OPS-06	Model service coverage not enforced in CI	Low	Add coverage gates for model-service-1 and model-service-2

C.5 Governance Gaps

ID	Gap	Priority	Recommendation
GOV-01	No change advisory board process documented	Medium	Document change approval process for production deployments
GOV-02	No data classification policy	Medium	Classify data types (PII, sensitive, public) and document handling requirements
GOV-03	No third-party dependency governance	Low	Document policy for evaluating and approving third-party dependencies
GOV-04	No model governance / ML lifecycle documentation	Medium	Document model versioning, validation, and deployment approval process
GOV-05	No accessibility compliance evidence	Low	Conduct WCAG audit if required by target markets

---

This document was generated from a comprehensive repository audit conducted on 19 April 2026. All claims are traceable to repository evidence as noted throughout. This document should be reviewed and updated whenever significant architectural changes are made to the platform.

COMPILED SOURCE DOCUMENT

Architecture Diagrams

`documentation/ARCHITECTURE_DIAGRAM.md`

Rendered visual appendix covering system design, runtime topology, environment layout, and the delivery and promotion model.

> **Note:** For the canonical auditor-ready architecture documentation, see ARCHITECTURE_OVERVIEW.md. This diagram document is retained as a supporting visual reference and remains accurate.
>
> **Rendered output:** This file is the Mermaid source. If your Markdown viewer does not render Mermaid, generate the rendered outputs with `npm run docs:arch:html` or `npm run docs:arch:pdf`.

This document refreshes the Talenti diagrams around the Azure-first platform that is now encoded in the repository. It separates the system into four views:

- high-level system design
- runtime topology
- environment and infrastructure layout
- delivery and promotion flow

Source Of Truth

These diagrams are based on:

- `infra/modules/platform.bicep`
- `infra/dev/main.bicep`
- `infra/uat/main.bicep`
- `infra/prod/main.bicep`
- `pr-fast-quality.yml`
- `pr-security-iac.yml`
- `pr-ephemeral-deploy.yml`
- `ci-main.yml`
- `deploy-dev.yml`
- `release.yml`
- `promote-release.yml`
- `backend/app/main.py`
- `backend/app/services/job_handlers.py`

The older local `docker-compose` setup remains useful for development, but it is no longer the primary source of truth for deployed architecture.

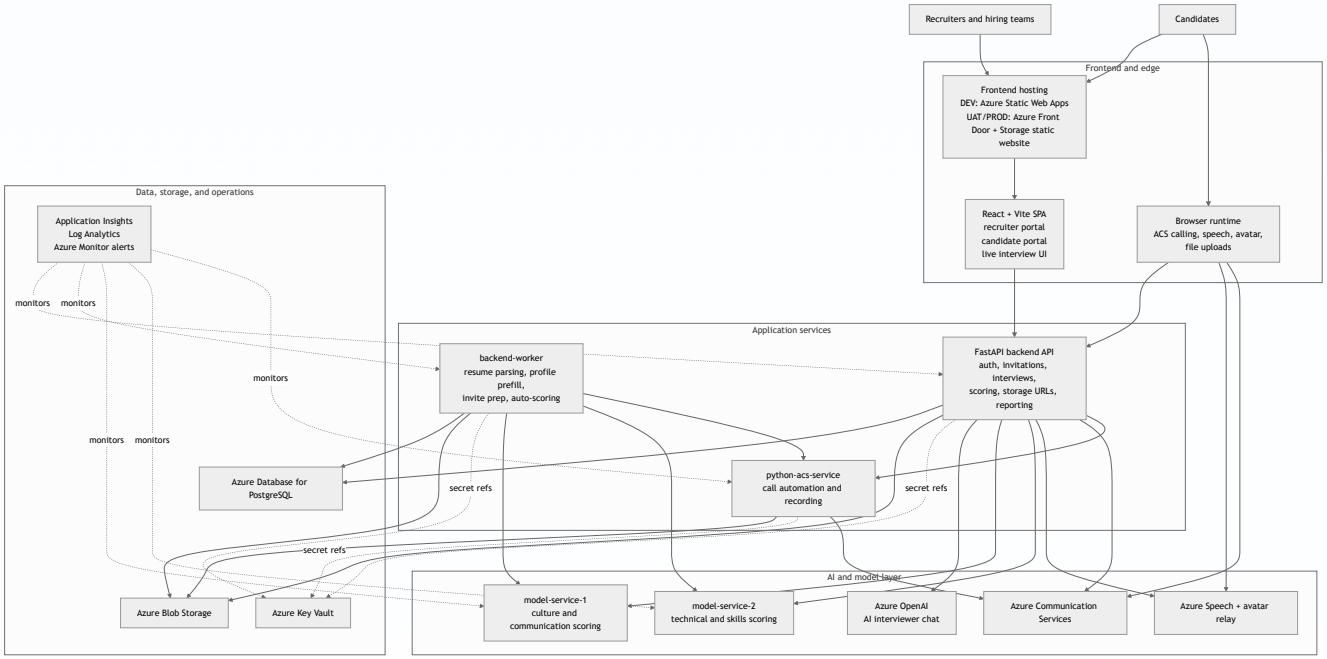
1. High-Level System Design

This is the quickest view to use when we want to explain the platform in one diagram.

ARCHITECTURE DIAGRAMS

1. High-Level System Design

documentation/ARCHITECTURE_DIAGRAM.md



System Design Notes

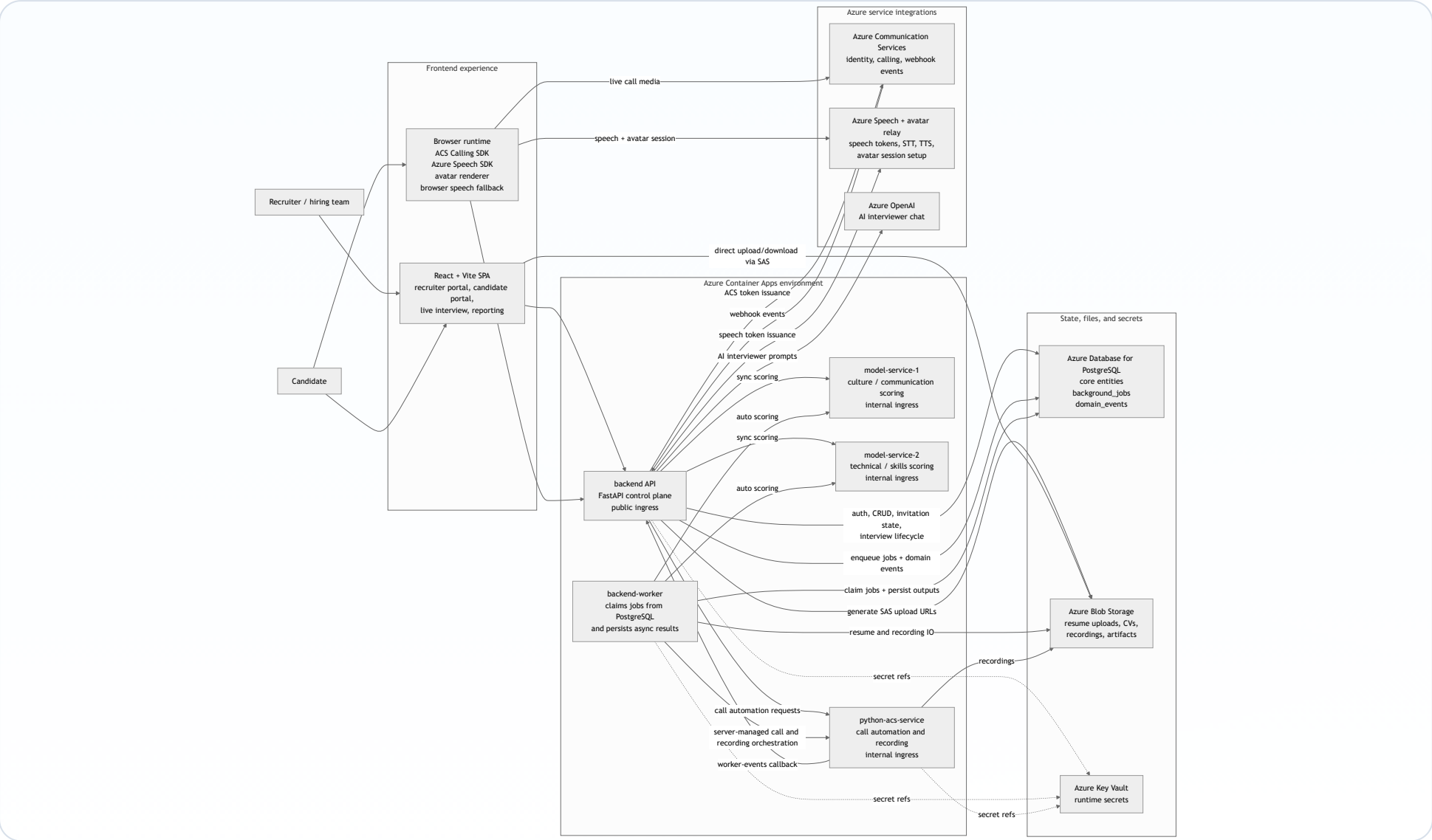
- The frontend is a single SPA, but the hosting pattern changes by environment: Static Web Apps in `dev`, Storage static website plus Front Door in `uat` and `prod`.
- The backend is the control plane for platform state, while `backend-worker` handles database-backed async orchestration.
- The model services remain separate runtime resources so scoring can evolve independently from the main backend.
- Azure Speech, avatar features, and Azure Communication Services are used both through backend-issued tokens and direct browser sessions.
- PostgreSQL, Blob Storage, Key Vault, and Azure-native monitoring form the shared platform foundation across all environments.

2. Runtime Topology

ARCHITECTURE DIAGRAMS

2. Runtime Topology

documentation/ARCHITECTURE_DIAGRAM.md



Runtime Notes

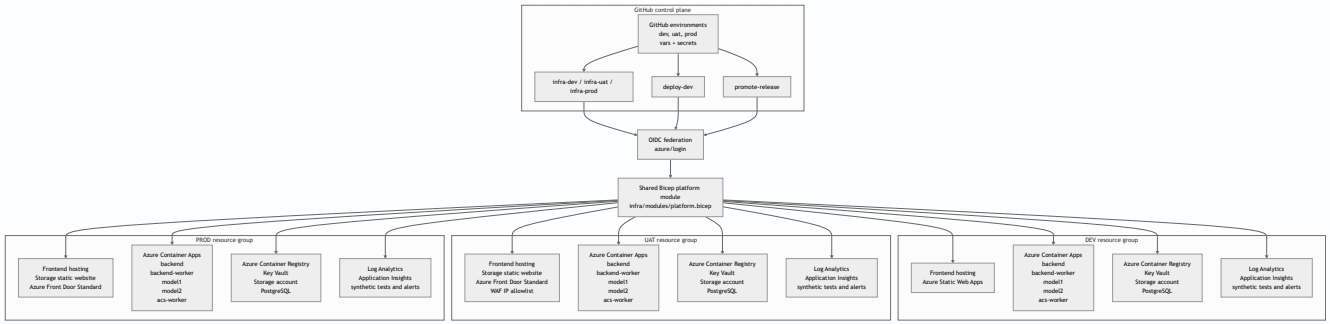
- Only `backend` has public ingress in the deployed Container Apps environment.
- `backend-worker`, `model-service-1`, `model-service-2`, and `python-acs-service` are internal services.
- Async orchestration is database-backed through `background_jobs` and `domain_events`; there is no separate queue broker in the current platform.
- Blob Storage is the canonical deployed upload and recording store. `/api/v1/candidates/cv` remains only as a local fallback when blob configuration is absent.
- The backend can score interviews synchronously through `/api/v1/scoring/analyze`, while `backend-worker` can also run asynchronous scoring when `AUTO_SCORE_INTERVIEWS` is enabled.

3. Environment And Infrastructure Topology

ARCHITECTURE DIAGRAMS

3. Environment And Infrastructure Topology

documentation/ARCHITECTURE_DIAGRAM.md



Environment Notes

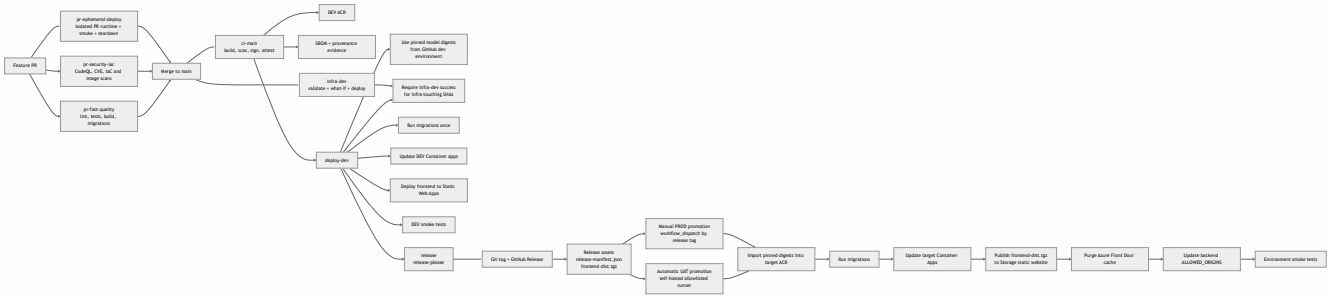
- Each stage is a separate Azure resource group with its own Container Apps environment, ACR, Key Vault, Storage account, PostgreSQL server, Log Analytics workspace, Application Insights component, and alert rules.
- `dev` serves the SPA from Azure Static Web Apps.
- `uat` and `prod` serve the SPA from Azure Storage static website hosting behind Azure Front Door Standard.
- `uat` adds a Front Door WAF IP allowlist so promotion and smoke checks must run from an allowlisted network.
- After infra deployment, the workflows assign `AcrPull` and `Key Vault Secrets User` to the five Container Apps so images and secret references work without embedded credentials.

4. Delivery And Promotion Architecture

ARCHITECTURE DIAGRAMS

4. Delivery And Promotion Architecture

documentation/ARCHITECTURE_DIAGRAM.md



Pipeline Notes

- Required PR gates are `pr-fast-quality`, `pr-security-iac`, and `pr-ephemeral-deploy`.
- `ci-main` builds backend and ACS worker images once per `main` SHA, then scans, signs, and attests those immutable artifacts.
- `deploy-dev` deploys immutable backend/ACS digests for the source SHA and enforces `infra-dev` success for infra-touching commits.
- `release.yml` is triggered only after successful `deploy-dev` on `main`, so release creation starts after DEV deployment validation.
- `release-manifest.json` is the promotion handoff. It captures backend, ACS worker, and model image digests plus the frontend source SHA.
- `promote-release.yml` does not rebuild application artifacts for UAT or PROD. It imports pinned images into the target ACR, verifies signatures, and reuses the packaged frontend artifact.
- UAT is auto-promoted from a published release. PROD is promoted manually by release tag.

Summary

The platform is now best understood as an Azure Container Apps control plane with PostgreSQL-backed orchestration, Blob-backed file storage, and GitHub Actions-managed promotion between isolated Azure environments. The key architectural distinction from the older diagrams is that deployment automation, environment isolation, and immutable release promotion are now first-class parts of the system design, not side notes.

COMPILED SOURCE DOCUMENT

Release Pipeline

`documentation/RELEASE_PIPELINE.md`

Detailed CI/CD control reference for branching, PR quality gates, release manifests, promotion rules, artifact integrity, and rollback mechanics.

Talenti now uses a trunk-based release model on `main`.

Branching and Merge Rules

- All feature and fix PRs target `main`.
- PR titles must follow Conventional Commits, for example:
  - `feat(api): add release manifest upload`
  - `fix(worker): emit queue metrics for alerting`
- Merge with squash so the PR title becomes the release note source.
- Configure branch protection in GitHub so:
  - `pr-fast-quality`, `pr-security-iac`, and `pr-ephemeral-deploy` are required before merge
  - `CodeQL Advanced` is enabled as the sole CodeQL workflow to avoid duplicate alerts and stale statuses
  - squash merge is allowed
  - direct pushes to `main` are blocked
- Configure GitHub Actions workflow permissions so:
  - default workflow permissions are `Read`
  - `Allow GitHub Actions to create and approve pull requests` is disabled

Workflows

- `pr-fast-quality.yml`
  - Runs on pull requests to `main`
  - Enforces Conventional Commit PR titles
  - Runs frontend lint/test/build, backend tests, ACS tests, migration execution checks, and coverage gates
- `pr-security-iac.yml`
  - Runs on pull requests to `main`
  - Runs secret scanning, dependency auditing, container scanning, and Bicep/IaC policy checks
- `codeql.yml`
  - Runs on pull requests to `main`, pushes to `main`, and a weekly schedule
  - Is the single CodeQL workflow for Actions, JavaScript/TypeScript, and Python analysis
- `pr-ephemeral-deploy.yml`
  - Runs on pull requests to `main` from internal branches
  - Uses the dedicated GitHub `pr-dev` environment so `dev` can remain restricted to `main`
  - Provisions ephemeral PR runtime resources, runs migrations and API smoke tests, then tears everything down
- `ci-main.yml`
  - Runs on pushes to `main`
  - Builds backend/ACS images once per SHA
  - Publishes immutable image digests
  - Generates SBOMs, vulnerability scan evidence, signatures, and provenance attestations
- `deploy-dev.yml`
  - Runs after a successful `ci-main` workflow on `main`
  - Manual dispatch requires `source_sha` (full 40-character SHA on `main`)
  - Manual dispatch validates a successful `ci-main` run exists for that SHA before deployment
  - Resolves immutable backend/ACS image digests from ACR for the commit SHA
  - Validates pinned model digests
  - Requires matching `infra-dev` success when infra files changed in that commit
  - Runs migrations once
  - Deploys DEV backend, worker, ACS worker, and frontend
  - Smoke-checks backend `/health` and the DEV Static Web App
- `release.yml`
  - Runs after successful `deploy-dev` on `main` (or manual dispatch)
  - Manual dispatch requires `source_sha` and validates that SHA was already deployed to `dev`
  - Uses `release-please` with repo-root `VERSION` and `CHANGELOG.md`
  - Creates or updates the release PR
  - When the release PR is merged and a tag is created, uploads:
    - `release-manifest.json`
    - `frontend-dist.tgz`
- `promote-release.yml`

    - Auto-promotes published releases to UAT
    - Allows manual promotion to UAT or PROD by release tag
    - Imports immutable image digests into the target ACR when needed
    - Verifies backend/ACS release signatures before deployment
    - Reuses the release frontend artifact instead of rebuilding it
    - Publishes UAT/PROD frontend assets to Azure Storage static website hosting
    - Purges Azure Front Door Standard cache after upload
    - Runs UAT on a self-hosted runner inside the allowlisted network
    - Optionally verifies model image signatures when `MODEL1_COSIGN_IDENTITY_REGEX` and `MODEL2_COSIGN_IDENTITY_REGEX` are set in the target environment

- Workflow security baseline

    - Third-party GitHub Actions are pinned to immutable commit SHAs.
    - Release tooling binaries (Syft, Trivy, Hadolint, Gitleaks) are downloaded at fixed versions with checksum verification.

Release Contract

- Repo-level version source: `VERSION`
- Human changelog: `CHANGELOG.md`
- Git tag format: `vX.Y.Z`
- Machine promotion contract: `release-manifest.json`

`release-manifest.json` contains:

- `version`
- `git_sha`
- `backend_image`
- `acs_worker_image`
- `model1_image`
- `model2_image`
- `frontend_source_sha`
- `created_at`

GitHub Environment Variables and Secrets

Create `pr-dev`, `dev`, `uat`, and `prod` GitHub environments with these variables:

- `AZURE_LOCATION`
- `AZURE_RESOURCE_GROUP`
- `ACR_NAME`
- `KEY_VAULT_NAME`
- `BACKEND_APP`
- `BACKEND_WORKER_APP`
- `MODEL1_APP`
- `MODEL2_APP`
- `ACS_WORKER_APP`
- `ALERT_EMAIL_ADDRESS` in `dev`, `uat`, and `prod`
- `STATIC_WEB_APP_NAME` in `dev` only
- `MODEL1_IMAGE_REF` and `MODEL2_IMAGE_REF` in `dev` only
- `FRONTEND_STORAGE_ACCOUNT` in `uat` and `prod`
- `FRONT_DOOR_PROFILE_NAME` in `uat` and `prod`
- `FRONT_DOOR_ENDPOINT_NAME` in `uat` and `prod`
- `FRONTEND_ALLOWED_CIDRS` in `uat` only, as a JSON array string such as `["203.0.113.0/24"]`
- `CONTAINER_ENV_NAME`, `POSTGRES_SERVER_NAME`, and `STORAGE_ACCOUNT_NAME` in `pr-dev`

Store these secrets per environment:

- `AZURE_CLIENT_ID`
- `AZURE_TENANT_ID`
- `AZURE_SUBSCRIPTION_ID`
- `POSTGRES_ADMIN_PASSWORD`
- `BACKEND_DATABASE_URL`
- `JWT_SECRET`

`pr-dev` needs `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`, `BACKEND_DATABASE_URL`, and `JWT_SECRET`. It does not require `POSTGRES_ADMIN_PASSWORD` because the PR workflow reuses the shared dev server and creates isolated databases from `BACKEND_DATABASE_URL`.

Optional repository or environment secret:

- `RELEASE_PLEASE_TOKEN`

Additional `pr-dev` variables for the PR ephemeral deploy gate:

- `CONTAINER_ENV_NAME` (defaults to `cae-talenti-dev-aue` if omitted)
- `POSTGRES_SERVER_NAME` (defaults to `psql-talenti-dev-aue` if omitted)
- `STORAGE_ACCOUNT_NAME` (defaults to `sttalentidevaue` if omitted)

Use a PAT for `RELEASE_PLEASE_TOKEN` if you want downstream workflows to respond to release publication automatically. Without it, manual promotion still works. The repository-level GitHub Actions workflow permissions above are still required so `release.yml` can create or update the release PR with `release-please`.

To create the environments and seed their OIDC secrets from the command line, use:

```
.\scripts\setup-deployment-access.ps1 -SubscriptionId <sub-id> -AlertEmailAddress <team-email>
```

That setup script also configures the repository GitHub Actions workflow permission model needed by `release.yml`.

The Azure federated credential subject must match the GitHub environment form used by the workflows:

- `repo:dcava30/Talenti_MVP:environment:dev`
- `repo:dcava30/Talenti_MVP:environment:pr-dev`
- `repo:dcava30/Talenti_MVP:environment:uat`
- `repo:dcava30/Talenti_MVP:environment:prod`

Promotion and Rollback

- UAT is promoted from a published GitHub Release.
- PROD is promoted manually by release tag.
- Promotion always uses the manifest digests captured at release time.
- UAT frontend traffic is restricted at Azure Front Door Standard with an IP allowlist custom WAF rule.
- UAT promotion and frontend smoke checks require a self-hosted runner with the `uat` label on an allowlisted office/VPN network.
- Roll back by rerunning `promote-release.yml` with an older `release_tag`.

Local Pre-Commit Gate

To catch workflow failures before pushing:

1. Install the tracked git hook:
    - `npm run hooks:install`
2. Default commit hook runs:
    - `scripts/run-pre-commit.ps1 -Scope staged -Profile fast`
3. Run full local parity checks manually when needed:
    - `npm run precommit:full`
4. Optional tools used by `full` profile:
    - Docker (for gitleaks, hadolint, trivy image scans)
    - Azure CLI + Bicep (for infra compile checks)
5. Every run writes a timestamped log:
    - `.tmp/precommit/precommit-YYYYMMDD-HHMMSS.log`
6. Failures end with a summary that includes:
    - the failed step
    - why that step is a gate
    - targeted remediation hints

`fast` mirrors `pr-fast-quality` behavior for touched components. `full` adds local equivalents of `pr-security-iac` checks. CodeQL remains CI-only in `codeql.yml`. Backend coverage gate is currently enforced at 60% in local and CI while test coverage is being expanded. ACS coverage gate is currently enforced at 40% in local and CI while ACS test coverage is being expanded. If Git shell (`sh.exe`) is broken on Windows, `hooks:install` auto-falls back to `.githooks-disabled` and prompts manual `npm run precommit`. If local ACS Python is not 3.11-compatible, pre-commit automatically runs ACS checks in `python:3.11-slim` Docker runtime for CI parity.

Environment Protection Notes

- `pr-dev` should use a deployment branch policy that allows `refs/pull/*/merge`.
- `dev` should use a deployment branch policy restricted to `main`.
- `uat` and `prod` should use required reviewers when the billing plan supports environment reviewer rules.
- If reviewer protection is not supported on the current plan, keep `can_admins_bypass=false` and enforce approvals via repository process until the plan is upgraded.

COMPILED SOURCE DOCUMENT

Deployment Runbook

`documentation/DEPLOYMENT_DEV_V2.md`

Hands-on deployment topology and environment configuration reference for the DEV estate and promotion-aligned release candidate flow.

Topology

- Public:
  - Frontend (Static Web Apps in `eastasia`; all core DEV infra remains in `australiaeast`)
  - Backend API (Container Apps external ingress)
- Internal only:
  - backend-worker (`python -m app.worker_main`)
  - model-service-1
  - model-service-2
  - acs-worker (`python-acs-service`)

Database

- Backend is the only system-of-record.
- Use PostgreSQL database: `talenti_backend_dev`.
- Backend worker and ACS worker both use backend-managed state; neither requires a separate database.

Required Backend App Settings

- `DATABASE_URL`
- `JWT_SECRET`
- `ALLOWED_ORIGINS`
- `MODEL_SERVICE_1_URL`
- `MODEL_SERVICE_2_URL`
- `ACS_WORKER_URL`
- `ACS_WORKER_SHARED_SECRET`
- `PUBLIC_BASE_URL`
- `LOG_LEVEL`
- Existing Azure settings:
  - `AZURE_ACS_CONNECTION_STRING`
  - `AZURE_SPEECH_KEY`
  - `AZURE_SPEECH_REGION`
  - `AZURE_OPENAI_ENDPOINT`
  - `AZURE_OPENAI_API_KEY`
  - `AZURE_OPENAI_DEPLOYMENT`
  - `AZURE_STORAGE_ACCOUNT`
  - `AZURE_STORAGE_ACCOUNT_KEY`
  - `AZURE_STORAGE_CONTAINER`

Required Backend Worker App Settings

- `DATABASE_URL`
- `JWT_SECRET`
- `ENVIRONMENT`
- `MODEL_SERVICE_1_URL`
- `MODEL_SERVICE_2_URL`
- `ACS_WORKER_URL`
- `ACS_WORKER_SHARED_SECRET`
- `PUBLIC_BASE_URL`
- `AZURE_STORAGE_ACCOUNT`
- `AZURE_STORAGE_ACCOUNT_KEY`
- `AZURE_STORAGE_CONTAINER`
- `BACKGROUND_WORKER_POLL_INTERVAL_SECONDS`
- `BACKGROUND_WORKER_METRICS_LOG_INTERVAL_SECONDS`
- `AUTO_SCORE_INTERVIEWS`
- `LOG_LEVEL`

Required ACS Worker App Settings

- `ACS_CONNECTION_STRING`
- `AZURE_STORAGE_CONNECTION_STRING`
- `RECORDING_CONTAINER=recordings`
- `BACKEND_INTERNAL_URL`
- `ACS_WORKER_SHARED_SECRET`
- `ACS_CALLBACK_URL` (set to backend public callback endpoint)

CI/CD

- `pr-fast-quality.yml` : validates PR title, runs frontend/backend/ACS checks, migration execution checks, and coverage gates.
- `pr-security-iac.yml` : enforces secret scanning, dependency/security scanning, container scanning, and Bicep/IaC policy checks.
- `pr-ephemeral-deploy.yml` : deploys isolated PR runtime resources in Azure using the dedicated GitHub `pr-dev` environment, runs migrations and API smoke checks, and tears down resources.
- `ci-main.yml` : builds immutable backend/ACS images once per `main` SHA, generates SBOMs, scans, signatures, and provenance attestations.
- `infra-dev.yml` : validates Bicep and IaC policy on PR/push, runs what-if, and deploys DEV Bicep infra on `main` .
- `codeql.yml` : is the repository's sole CodeQL workflow for PRs, `main` , and weekly scheduled scans.
- `deploy-dev.yml` :
  - Runs after successful `ci-main` on `main` .
  - Resolves immutable backend/ACS image digests for the exact source SHA.
  - Deploys the same backend image digest to both `backend` and `backend-worker` .
  - Consumes pinned model images from GitHub environment `dev` variables:
    - `MODEL1_IMAGE_REF`
    - `MODEL2_IMAGE_REF`
  - Fails fast if model refs are missing, malformed, or unresolved in ACR.
  - Requires matching `infra-dev` success for infra-touching commits.
  - Preflight-checks the resource group, ACR, Key Vault, Container Apps, and Static Web App.
  - Runs backend migrations once, deploys internal services then backend, deploys frontend, and smoke-checks backend/frontend availability.
- `release.yml` :
  - Runs only after successful `deploy-dev` for `main` (or manual dispatch).
  - Uses `release-please` to manage repo-root `VERSION` , `CHANGELOG.md` , and GitHub Releases.
  - Uploads `release-manifest.json` and `frontend-dist.tgz` for promotion.
- `promote-release.yml` :
  - Promotes the released digests and frontend artifact to UAT/PROD without rebuilding them.
  - Verifies release image signatures before deployment.
  - Uploads the frontend artifact to Azure Storage static website hosting and purges Azure Front Door Standard cache.
  - Uses a self-hosted `uat` runner for restricted UAT frontend smoke tests.

Seed the GitHub `dev` and `pr-dev` environments and Azure OIDC identities with:

```
.\scripts\setup-deployment-access.ps1 -SubscriptionId <sub-id> -EnvironmentNames dev,pr-dev -Al
```

For first-time DEV runtime bootstrap, use:

```
.\scripts\day1-dev-deploy.ps1 -SubscriptionId <sub-id> -StaticWebAppLocation eastasia
```

Pinned Model Promotion Runbook

1. In model repo pipelines, retrieve immutable digest refs for successful dev images:
    - `acrtalentidev.azurecr.io/talenti/model-service-1@sha256:<64-hex>`
    - `acrtalentidev.azurecr.io/talenti/model-service-2@sha256:<64-hex>`
2. In GitHub -> main repo -> Settings -> Environments -> `dev`, set variables:
    - `MODEL1_IMAGE_REF=<model1 digest ref>`
    - `MODEL2_IMAGE_REF=<model2 digest ref>`
3. Trigger `deploy-dev` workflow (manual or matching push).
4. Verify deployment:
    - Backend health endpoint returns 200.
    - `backend-worker` is running and `background_jobs` are not stuck in `pending`.
    - Container Apps `ca-model1-dev` and `ca-model2-dev` revisions reference the pinned digest images.

GitHub Environment Variables

Set these variables in the GitHub `dev` environment:

- `AZURE_LOCATION`
- `AZURE_RESOURCE_GROUP`
- `ACR_NAME`
- `KEY_VAULT_NAME`
- `STATIC_WEB_APP_NAME`
- `BACKEND_APP`
- `BACKEND_WORKER_APP`
- `MODEL1_APP`
- `MODEL2_APP`
- `ACS_WORKER_APP`
- `ALERT_EMAIL_ADDRESS`
- `MODEL1_IMAGE_REF`
- `MODEL2_IMAGE_REF`

Set these secrets in the GitHub `dev` environment:

- `AZURE_CLIENT_ID`
- `AZURE_TENANT_ID`
- `AZURE_SUBSCRIPTION_ID`
- `POSTGRES_ADMIN_PASSWORD`
- `BACKEND_DATABASE_URL`
- `JWT_SECRET`

Migration

- Backend runs Alembic `upgrade head` automatically at startup and fails fast on migration errors.
- Keep `python backend/scripts/run_migrations.py` in the deployment pipeline as a pre-deploy safety check.
- If `backend-worker` is not deployed or unhealthy, interview orchestration and async file/scoring jobs will remain queued in `background_jobs`.

COMPILED SOURCE DOCUMENT

Monitoring and Observability

`documentation/MONITORING.md`

Operational monitoring baseline covering health, alerting, logs, troubleshooting, and incident support expectations.

> Last Updated: April 2026
>
> **See also:** ARCHITECTURE_OVERVIEW.md, Section 17 for the architectural context of monitoring and observability.

Talenti runs as a set of FastAPI microservices (backend API, backend-worker, model-service-1, model-service-2, python-acs-service) backed by PostgreSQL, Azure integrations, and GitHub Actions deployments. The platform assumes Azure-native observability with Log Analytics, Application Insights, Azure Monitor alerts, and GitHub Release metadata.

What to Monitor

API Health

- Backend API: `GET /health` on port 8000.
- model-service-1: `GET /health` on port 8001.
- model-service-2: `GET /health` on port 8002.
- python-acs-service: `GET /health`, `GET /health/ready`, `GET /health/live` (internal).
- HTTP 5xx error rate on `/api/*` routes.
- Latency percentiles (p50/p95/p99).
- Presence of `X-Request-ID` on backend responses for incident traceability.

Database (PostgreSQL)

- Database connectivity and server health for PostgreSQL.
- Disk usage and available capacity.
- Migration status on startup (Alembic).
- Queue depth and job age in `background_jobs`.
- Event volume and write failures in `domain_events`.

Azure Services

- Azure Communication Services token issuance failures.
- Azure Speech token issuance failures.
- Azure OpenAI request errors/timeouts.
- Azure Blob Storage upload URL generation failures.

Workers & Model Services

- `backend-worker` process health and restart count.
- Structured queue metrics emitted by `backend-worker`:
  - `pending_jobs`
  - `oldest_pending_job_age_seconds`
- `python-acs-service` readiness (ACS connection, storage, backend callback connectivity).
- ACS worker callback failures (`/api/v1/acs/worker-events`).
- model-service-1 and model-service-2 prediction latency and error rate.
- Model service container restart count and memory usage (model loading can require 2-4GB).

Logs

- Backend API and ACS worker emit structured JSON logs to stdout.
- Backend request middleware adds `X-Request-ID` and includes `request_id`, route, method, status code, and duration in request-completion logs.
- Worker logs include `correlation_id`, `job_type`, `job_id`, and queue-metric events.
- In Azure Container Apps, these logs flow to Log Analytics and are used by scheduled query alerts.

Azure Alerting Baseline

Each environment should have:

- Log Analytics workspace
- Application Insights component
- Azure Monitor Action Group with a shared team email
- Synthetic availability checks for:
  - frontend root URL
  - backend `/health`
- Metric alerts for:
  - backend 5xx spikes
  - backend latency degradation
  - container restart spikes
- Scheduled query alerts for:
  - frontend synthetic failure
  - backend synthetic failure
  - background job backlog
  - background job age

Alerts

- **Backend Health Down:** backend availability test fails.
- **Frontend Health Down:** frontend availability test fails.
- **Backend 5xx Spike:** container app request metric shows sustained 5xx traffic.
- **Latency Regression:** backend response-time metric exceeds the configured threshold.
- **Container Restarts:** restart count exceeds the configured threshold on backend, worker, model, or ACS apps.
- **Job Backlog:** `pending_jobs` from worker queue metrics exceeds the configured threshold.
- **Job Age:** `oldest_pending_job_age_seconds` exceeds the configured threshold.

Troubleshooting Checklist

1. Check FastAPI logs for stack traces.
2. Check `backend-worker` logs if orchestration, CV post-processing, or async scoring is stalled.
3. Check model-service-1/model-service-2 logs if scoring returns errors or timeouts. Verify `MODEL_SERVICE_1_URL` and `MODEL_SERVICE_2_URL` are reachable from the backend.
4. Check `python-acs-service` logs if call automation or recordings fail. Verify `ACS_WORKER_SHARED_SECRET` matches between backend and ACS service.
5. Use the `X-Request-ID` response header to correlate API failures to backend logs in Log Analytics.
6. Verify `.env` or Container App configuration for Azure keys, `DATABASE_URL`, `BACKGROUND_WORKER_POLL_INTERVAL_SECONDS`, `BACKGROUND_WORKER_METRICS_LOG_INTERVAL_SECONDS`, and `AUTO_SCORE_INTERVIEWS`.
7. Confirm Azure Monitor alert state, web test health, and Action Group delivery.
8. Confirm PostgreSQL connectivity, storage capacity, and server health.
9. Verify Azure service health and quotas.

COMPILED SOURCE DOCUMENT

API Reference

documentation/API_REFERENCE.md

Endpoint inventory for auditors who need to inspect exposed capabilities, authentication boundaries, audit-log endpoints, and platform surface area.

> Last Updated: April 2026

This API is served by the FastAPI backend on port 8000 and uses PostgreSQL for persistence. All endpoints require a valid JWT unless explicitly noted. Tokens are passed via `Authorization: Bearer <token>`.

Base URL

```
http://<backend-host>:8000
```

Interactive docs are available at `/docs` (Swagger UI) and `/redoc` (ReDoc) in non-production environments.

Health

Method	Endpoint	Description	Auth
GET	`/health`	Service health check	No

Authentication (`/api/auth`)

Method	Endpoint	Description	Auth
POST	`/api/auth/register`	Create a new user account	No
POST	`/api/auth/login`	Obtain access and refresh tokens	No
POST	`/api/auth/refresh`	Refresh an access token using the refresh-token cookie	No
POST	`/api/auth/logout`	Revoke the current session	No
GET	`/api/auth/me`	Get the current authenticated user	Yes
GET	`/api/auth/claim-context`	Validate an invitation token before account claim (query: `token`)	No
POST	`/api/auth/claim-invite`	Claim an invited account with email and password	No

---

Organizations (`/api/orgs`)

Method	Endpoint	Description	Auth
POST	`/api/orgs`	Create a new organization	Yes
GET	`/api/orgs/current`	Get current user's organization membership	Yes
PATCH	`/api/orgs/{organisation_id}/retention`	Update recording retention policy	Yes
GET	`/api/orgs/{organisation_id}/stats`	Get org statistics (roles, candidates, interviews, avg score)	Yes
POST	`/api/orgs/{organisation_id}/environment`	Submit operating-environment questionnaire (10 answers)	Yes
POST	`/api/orgs/{organisation_id}/environment/aggregate`	Aggregate environment from multiple respondents via majority voting	Yes

---

Job Roles (`/api/roles`)

Method	Endpoint	Description	Auth
POST	`/api/roles`	Create a job role	Yes
GET	`/api/roles`	List job roles (optional query: `organisation_id`)	Yes
GET	`/api/roles/{role_id}`	Get job role details	Yes
PATCH	`/api/roles/{role_id}`	Update job role fields	Yes
PATCH	`/api/roles/{role_id}/rubric`	Update scoring rubric for a role	Yes

Requirements Extraction (`/api/v1/roles`)

Method	Endpoint	Description	Auth
POST	`/api/v1/roles/extract-requirements`	Extract skills, responsibilities, and qualifications from a job description	Yes

---

Candidates (`/api/v1/candidates`)

Profile

Method	Endpoint	Description	Auth
GET	`/api/v1/candidates/profile`	Get candidate profile (optional query: `user_id`)	Yes
POST	`/api/v1/candidates/profile`	Upsert (create or update) candidate profile	Yes
PATCH	`/api/v1/candidates/{user_id}/profile`	Update candidate profile by user ID	Yes
POST	`/api/v1/candidates/profile/confirm`	Confirm and lock candidate profile	Yes
POST	`/api/v1/candidates/parse-resume`	Parse resume text or uploaded file	Yes
POST	`/api/v1/candidates/cv`	Upload CV file directly (local-development fallback; use `/api/storage/upload-url` in deployed environments)	Yes
DELETE	`/api/v1/candidates/{user_id}`	Delete candidate account	Yes

Employment History

Method	Endpoint	Description	Auth
GET	`/api/v1/candidates/employment`	List employment history (optional query: `user_id`)	Yes
POST	`/api/v1/candidates/employment`	Create employment history entry	Yes
PATCH	`/api/v1/candidates/employment/{employment_id}`	Update employment entry	Yes
DELETE	`/api/v1/candidates/employment/{employment_id}`	Delete employment entry	Yes

Education

Method	Endpoint	Description	Auth
GET	`/api/v1/candidates/education`	List education entries (optional query: `user_id`)	Yes
POST	`/api/v1/candidates/education`	Create education entry	Yes
PATCH	`/api/v1/candidates/education/{education_id}`	Update education entry	Yes
DELETE	`/api/v1/candidates/education/{education_id}`	Delete education entry	Yes

Skills

Method	Endpoint	Description	Auth
GET	`/api/v1/candidates/skills`	List candidate skills (optional query: `user_id`)	Yes
POST	`/api/v1/candidates/skills`	Create skill entry	Yes
DELETE	`/api/v1/candidates/skills/{skill_id}`	Delete skill	Yes

Candidate-Facing Lists

Method	Endpoint	Description	Auth
GET	`/api/v1/candidates/applications`	List candidate's applications	Yes
POST	`/api/v1/candidates/applications`	Create application (body: `job_role_id`)	Yes
GET	`/api/v1/candidates/invitations`	List candidate's invitations	Yes
GET	`/api/v1/candidates/feedback`	List candidate feedback	Yes

Practice Interviews

Method	Endpoint	Description	Auth
GET	`/api/v1/candidates/practice-interviews`	List practice interviews	Yes
POST	`/api/v1/candidates/practice-interviews`	Create practice interview	Yes
GET	`/api/v1/candidates/practice-interviews/{practice_id}`	Get practice interview	Yes
PATCH	`/api/v1/candidates/practice-interviews/{practice_id}`	Update practice interview	Yes

Data Deletion (GDPR)

Method	Endpoint	Description	Auth
GET	`/api/v1/candidates/deletion-requests`	List data deletion requests	Yes
POST	`/api/v1/candidates/deletion-requests`	Create data deletion request	Yes

Resume Batches (`/api/v1/resume-batches`)

Bulk resume upload, parsing, and candidate invitation workflow.

Method	Endpoint	Description	Auth
POST	`/api/v1/resume-batches`	Create a new resume ingestion batch	Yes
GET	`/api/v1/resume-batches/{batch_id}`	Get batch details	Yes
GET	`/api/v1/resume-batches/{batch_id}/items`	List items in a batch	Yes
POST	`/api/v1/resume-batches/{batch_id}/items/upload-url`	Generate Blob Storage upload URL for a batch item	Yes
POST	`/api/v1/resume-batches/{batch_id}/process`	Process batch items (parse resumes via background jobs)	Yes
PATCH	`/api/v1/resume-batches/items/{item_id}`	Update batch item (email, review status)	Yes
POST	`/api/v1/resume-batches/{batch_id}/invite`	Send invitations to candidates from a processed batch	Yes

---

Applications (`/api/v1`)

Method	Endpoint	Description	Auth
POST	`/api/v1/applications`	Create application	Yes
GET	`/api/v1/applications`	List applications (optional query: `candidate_id`)	Yes
PATCH	`/api/v1/applications/{application_id}`	Update application status or metadata	Yes
GET	`/api/v1/applications/{application_id}/context`	Get full application context (candidate, job, org, competencies)	Yes
GET	`/api/v1/roles/{role_id}/applications`	List applications for a specific role	Yes

Invitations

Method	Endpoint	Description	Auth
POST	`/api/invitations`	Create invitation for an application	Yes
GET	`/api/v1/invitations`	List invitations (optional query: `application_id`)	Yes
PATCH	`/api/v1/invitations/{invitation_id}`	Update invitation status	Yes
GET	`/api/v1/invitations/validate`	Validate invitation token (query: `token`)	No

Interviews (`/api/v1/interviews`)

Method	Endpoint	Description	Auth
POST	`/api/v1/interviews`	Create interview session	Yes
POST	`/api/v1/interviews/start`	Start or resume the active interview for an application; triggers background orchestration	Yes
GET	`/api/v1/interviews/active`	Get active interview (query: `application_id`)	Yes
GET	`/api/v1/interviews/{interview_id}`	Get interview details	Yes
PATCH	`/api/v1/interviews/{interview_id}`	Update interview metadata	Yes
POST	`/api/v1/interviews/{interview_id}/complete`	Complete an interview; enqueues post-interview orchestration	Yes
GET	`/api/v1/interviews/{interview_id}/transcripts`	List transcript segments	Yes
POST	`/api/v1/interviews/{interview_id}/transcripts`	Add transcript segment	Yes
GET	`/api/v1/interviews/{interview_id}/score`	Get interview score	Yes
GET	`/api/v1/interviews/{interview_id}/dimensions`	List score dimensions	Yes
POST	`/api/v1/interviews/{interview_id}/scores`	Save or update interview scores	Yes
GET	`/api/v1/interviews/{interview_id}/report`	Get full interview report (interview + score + dimensions + transcripts)	Yes

---

AI Interview Chat (`/api/v1/interview`)

Method	Endpoint	Description	Auth
POST	`/api/v1/interview/chat`	AI interviewer chat — generates the next interview question	Yes

---

Scoring (`/api/v1/scoring`)

Method	Endpoint	Description	Auth
POST	`/api/v1/scoring/analyze`	Score interview transcript against culture fit (model-service-1) and skills fit (model-service-2)	Yes

---

Interview Scores (`/api/v1/interview-scores`)

Method	Endpoint	Description	Auth
PATCH	`/api/v1/interview-scores/{score_id}`	Update interview score	Yes
POST	`/api/v1/interview-scores/{score_id}/override`	Set human override of hiring recommendation	Yes
POST	`/api/v1/interview-scores/{score_id}/outcomes`	Record post-hire performance outcome	Yes

Shortlist (`/api/v1/shortlist`)

Method	Endpoint	Description	Auth
POST	`/api/v1/shortlist/generate`	Generate ranked shortlist of candidates for a role	Yes

---

Storage (`/api/storage`)

Method	Endpoint	Description	Auth
POST	`/api/storage/upload-url`	Generate Azure Blob Storage SAS upload URL for direct browser upload	Yes

This is the primary deployed CV/resume upload entrypoint. The flow is: request upload URL -> browser uploads directly to Blob Storage -> save `cv_file_id` on the candidate profile.

Azure Communication Services (`/api/v1/acs`)

Method	Endpoint	Description	Auth
POST	`/api/v1/acs/token`	Create ACS identity token for video calls	Yes
POST	`/api/v1/acs/webhook`	ACS webhook receiver for recording and call-state events	No
POST	`/api/v1/acs/worker-events`	Internal callback from python-acs-service for recording completion	Header (`X-ACS-Worker-Secret`)

Call Automation (`/api/v1/call-automation`)

Method	Endpoint	Description	Auth
POST	`/api/v1/call-automation/calls`	Create outbound interview call	Yes
POST	`/api/v1/call-automation/calls/{call_connection_id}/hangup`	Hang up an interview call	Yes
POST	`/api/v1/call-automation/recordings/start`	Start recording on an active call	Yes
POST	`/api/v1/call-automation/recordings/{recording_id}/stop`	Stop a recording	Yes

Speech (`/api/v1/speech`)

Method	Endpoint	Description	Auth
POST	`/api/v1/speech/token`	Generate Azure Speech Services token for TTS/STT	Yes

---

Data Retention (`/api/v1/data-retention`)

Method	Endpoint	Description	Auth
POST	`/api/v1/data-retention/cleanup`	Execute data retention cleanup (remove expired recordings, anonymize data)	Yes

---

Audit Log (`/api/v1/audit-log`)

Method	Endpoint	Description	Auth
GET	`/api/v1/audit-log`	List audit log entries (optional query: `organisation_id`)	Yes
POST	`/api/v1/audit-log`	Create audit log entry	Yes

Notes

- The backend uses PostgreSQL; configure `DATABASE_URL` in `.env`.
- Uploads are stored in Azure Blob Storage in deployed environments (see `.env.example`).
- Interview start/complete lifecycle endpoints trigger DB-backed background jobs processed by `backend-worker`.
- Scoring calls both model-service-1 (culture/behavioral fit) and model-service-2 (skills/technical fit) and combines results.
- The `python-acs-service` communicates with the backend via the `/api/v1/acs/worker-events` callback authenticated by a shared secret.
- All requests include an `X-Request-ID` header for tracing; pass one in the request or the backend generates a UUID.

COMPILED SOURCE DOCUMENT

Database Schema

`documentation/DATABASE_SCHEMA.md`

Core data architecture reference covering entity relationships, compliance-related records, access control notes, retention flows, and audit logging.

> **Version:** 1.1.0
> **Last Updated:** April 2026 **Database:** PostgreSQL

Overview

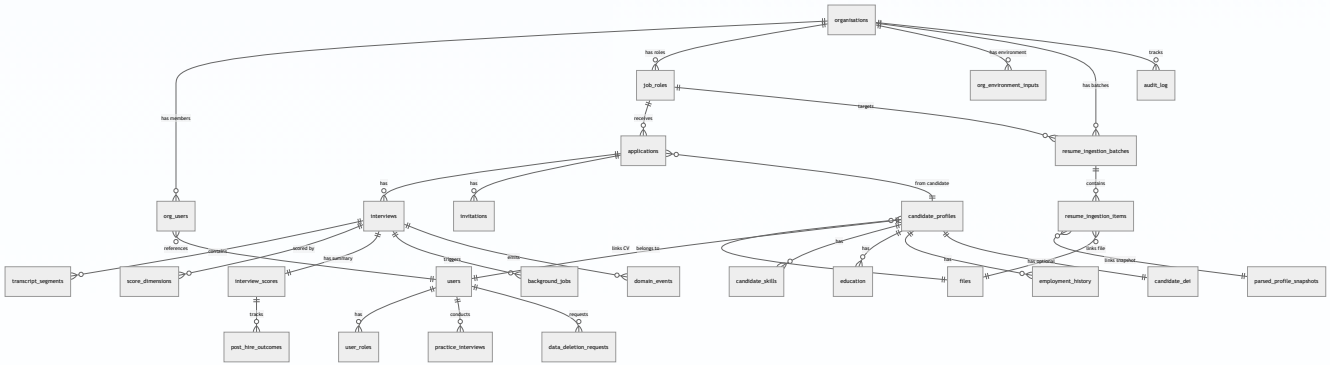
This document provides comprehensive documentation of the Talenti database schema, including table structures, relationships, access control policies, and data flow patterns.

Entity Relationship Diagram

DATABASE SCHEMA

Entity Relationship Diagram

documentation/DATABASE_SCHEMA.md



Tables

organisations

Company/business accounts that use the platform.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
name	text	NO	-	Organisation name
description	text	YES	NULL	About the organisation
industry	text	YES	NULL	Industry sector
website	text	YES	NULL	Company website URL
logo_url	text	YES	NULL	Logo image URL
billing_email	text	YES	NULL	Billing contact email
billing_address	text	YES	NULL	Billing address
values_framework	jsonb	YES	NULL	Company values for culture fit
recording_retention_days	integer	YES	NULL	How long to keep recordings
created_at	timestamptz	NO	now()	Creation timestamp
updated_at	timestamptz	NO	now()	Last update timestamp

Indexes:

- `organisations_pkey` (id)

---

org_users

Team members belonging to an organisation.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
organisation_id	uuid	NO	-	FK to organisations
user_id	uuid	NO	-	FK to users
role	text	NO	'member'	Role in org (admin, recruiter, viewer)
created_at	timestamptz	NO	now()	When added to org

Indexes:

- `org_users_pkey` (id)
- `org_users_org_user_unique` (organisation_id, user_id) UNIQUE

Foreign Keys:

- `org_users_organisation_id_fkey` - organisations(id)

---

job_roles

Open positions/job listings created by organisations.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
organisation_id	uuid	NO	-	FK to organisations
title	text	NO	-	Job title
description	text	YES	NULL	Full job description
department	text	YES	NULL	Department/team
location	text	YES	NULL	Work location
work_type	text	YES	NULL	remote/hybrid/onsite
employment_type	text	YES	NULL	full-time/part-time/contract
industry	text	YES	NULL	Industry category
salary_range_min	integer	YES	NULL	Minimum salary
salary_range_max	integer	YES	NULL	Maximum salary
requirements	jsonb	YES	NULL	Extracted requirements
scoring_rubric	jsonb	YES	NULL	Custom scoring weights
interview_structure	jsonb	YES	NULL	Interview configuration
status	job_role_status	NO	'draft'	Role status
created_by	uuid	YES	NULL	User who created
created_at	timestamptz	NO	now()	Creation timestamp
updated_at	timestamptz	NO	now()	Last update timestamp

Requirements JSONB Structure:

```
{
  "skills": ["React", "JavaScript"],
  "experience": ["5+ years frontend"],
  "qualifications": ["Bachelor's degree"],
  "responsibilities": ["Lead frontend team"]
}
```

Scoring Rubric JSONB Structure:

```
{
  "technical_skills": { "weight": 0.3, "label": "Technical Skills" },
  "communication": { "weight": 0.2, "label": "Communication" }
}
```

Indexes:

- `job_roles_pkey` (id)
- `job_roles_organisation_id_idx` (organisation_id)

Foreign Keys:

- `job_roles_organisation_id_fkey` - organisations(id)

---

candidate_profiles

Candidate personal information and profile data.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
user_id	uuid	NO	-	FK to users
first_name	text	YES	NULL	First name
last_name	text	YES	NULL	Last name
email	text	YES	NULL	Contact email
phone	text	YES	NULL	Phone number
suburb	text	YES	NULL	Suburb/city
state	text	YES	NULL	State/province
postcode	text	YES	NULL	Postal code
country	text	YES	NULL	Country
linkedin_url	text	YES	NULL	LinkedIn profile
portfolio_url	text	YES	NULL	Portfolio/website
cv_file_id	text	YES	NULL	FK to files.id for the linked CV
cv_file_path	text	YES	NULL	Path to CV in storage
cv_uploaded_at	timestamptz	YES	NULL	When CV was uploaded
availability	text	YES	NULL	When available to start
work_mode	text	YES	NULL	Preferred work mode
work_rights	text	YES	NULL	Work authorization
gpa_wam	numeric	YES	NULL	Academic score
profile_visibility	text	YES	NULL	Visibility setting
visibility_settings	jsonb	YES	NULL	Detailed visibility
paused_at	timestamptz	YES	NULL	When paused job search
created_at	timestamptz	NO	now()	Creation timestamp
updated_at	timestamptz	NO	now()	Last update timestamp

Indexes:

- `candidate_profiles_pkey` (id)
- `candidate_profiles_user_id_idx` (user_id) UNIQUE

---

candidate_skills

Skills listed by candidates.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
user_id	uuid	NO	-	FK to users
skill_name	text	NO	-	Skill name
skill_type	text	NO	-	hard/soft
proficiency_level	text	YES	NULL	beginner/intermediate/expert
created_at	timestamptz	NO	now()	Creation timestamp

Indexes:

- `candidate_skills_pkey` (id)
- `candidate_skills_user_id_idx` (user_id)

---

education

Educational history for candidates.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
user_id	uuid	NO	-	FK to users
institution	text	NO	-	School/university name
degree	text	NO	-	Degree title
field_of_study	text	YES	NULL	Major/field
start_date	date	YES	NULL	Start date
end_date	date	YES	NULL	End/graduation date
is_current	boolean	YES	false	Currently studying
created_at	timestamptz	NO	now()	Creation timestamp

Indexes:

- `education_pkey` (id)
- `education_user_id_idx` (user_id)

---

employment_history

Work history for candidates.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
user_id	uuid	NO	-	FK to users
company_name	text	NO	-	Employer name
job_title	text	NO	-	Position title
description	text	YES	NULL	Role description
start_date	date	NO	-	Start date
end_date	date	YES	NULL	End date (null if current)
is_current	boolean	YES	false	Currently employed
created_at	timestamptz	NO	now()	Creation timestamp

Indexes:

- `employment_history_pkey` (id)
- `employment_history_user_id_idx` (user_id)

---

candidate_dei

Diversity, equity, and inclusion data (optional, extra sensitive).

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
user_id	uuid	NO	-	FK to users
gender	text	YES	NULL	Gender identity
ethnicity	text	YES	NULL	Ethnic background
disability_status	text	YES	NULL	Disability disclosure
veteran_status	text	YES	NULL	Veteran status
created_at	timestamptz	NO	now()	Creation timestamp

Note: This data is used only for aggregate DEI reporting, never for individual candidate evaluation.

---

applications

Job applications from candidates.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
job_role_id	uuid	NO	-	FK to job_roles
candidate_id	uuid	NO	-	FK to users (candidate)
status	text	NO	'applied'	Application status
match_score	integer	YES	NULL	AI match score (0-100)
created_at	timestamptz	NO	now()	Application date
updated_at	timestamptz	NO	now()	Last update

Status Values:

- `applied` - Initial application
- `screening` - Under review
- `invited` - Invited to interview
- `interviewing` - Interview in progress
- `interviewed` - Interview completed
- `shortlisted` - On shortlist
- `rejected` - Not proceeding
- `hired` - Offered/accepted

Indexes:

- `applications_pkey` (id)
- `applications_job_role_id_idx` (job_role_id)
- `applications_candidate_id_idx` (candidate_id)

Foreign Keys:

- `applications_job_role_id_fkey` - job_roles(id)

---

interviews

Interview sessions.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
application_id	uuid	NO	-	FK to applications
status	interview_status	NO	'invited'	Interview status
started_at	timestamptz	YES	NULL	When interview began
ended_at	timestamptz	YES	NULL	When interview ended
duration_seconds	integer	YES	NULL	Total duration
call_connection_id	text	YES	NULL	ACS call connection identifier
server_call_id	text	YES	NULL	Server-managed ACS call identifier
recording_id	text	YES	NULL	Recording session identifier
recording_started	boolean	NO	false	Whether recording has started
recording_processed	boolean	NO	false	Whether recording has been processed
recording_status	text	YES	NULL	Recording lifecycle status
recording_error	text	YES	NULL	Recording failure message
recording_started_at	timestamptz	YES	NULL	When recording started
recording_stopped_at	timestamptz	YES	NULL	When recording stopped
recording_processed_at	timestamptz	YES	NULL	When recording was processed
recording_url	text	YES	NULL	URL to recording
transcript_status	text	YES	NULL	Transcript lifecycle status
anti_cheat_signals	text	YES	NULL	JSON-encoded detected anomalies
session_metadata	text	YES	NULL	JSON-encoded session/orchestration metadata
summary	text	YES	NULL	Interview summary text
created_at	timestamptz	NO	now()	Creation timestamp
updated_at	timestamptz	NO	now()	Last update

Session metadata JSON structure:

```
{
  "serverCallId": "...",
  "correlationId": "uuid",
  "lastEventAt": "2026-01-13T10:00:00Z"
}
```

Indexes:

- `interviews_pkey` (id)
- `interviews_application_id_idx` (application_id)

Foreign Keys:

- `interviews_application_id_fkey` - applications(id)

---

transcript_segments

Interview transcript broken into speaker segments.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
interview_id	uuid	NO	-	FK to interviews
speaker	text	NO	-	'ai' or 'candidate'
content	text	NO	-	Spoken text
start_time_ms	integer	NO	-	Start time in ms
end_time_ms	integer	YES	NULL	End time in ms
confidence	numeric	YES	NULL	STT confidence score
created_at	timestamptz	NO	now()	Creation timestamp

Indexes:

- `transcript_segments_pkey` (id)
- `transcript_segments_interview_id_idx` (interview_id)
- `transcript_segments_start_time_idx` (start_time_ms)

Foreign Keys:

- `transcript_segments_interview_id_fkey` - interviews(id)

---

files

Blob-backed uploaded files tracked by the backend.

Column	Type	Nullable	Default	Description
id	text	NO	uuid	Primary key
organisation_id	text	YES	NULL	Owning organisation when applicable
user_id	text	YES	NULL	Owning user when applicable
purpose	text	NO	'general'	File purpose such as `candidate_cv`
blob_path	text	NO	-	Blob path/key in storage
content_type	text	YES	NULL	MIME type
metadata	text	YES	NULL	JSON-encoded metadata
created_at	timestamptz	NO	now()	Creation timestamp

---

background_jobs

DB-backed asynchronous work queue processed by `backend-worker`.

Column	Type	Nullable	Default	Description
id	text	NO	uuid	Primary key
job_type	text	NO	-	Handler key such as `interview_start_orchestration`
status	text	NO	`pending`	`pending`, `running`, `completed`, `failed`, or `skipped`
payload_json	text	NO	-	JSON-encoded job payload
result_json	text	YES	NULL	JSON-encoded job result
attempts	integer	NO	0	Attempt count
max_attempts	integer	NO	3	Retry ceiling
available_at	timestamptz	NO	now()	Next eligible run time
started_at	timestamptz	YES	NULL	Processing start time
completed_at	timestamptz	YES	NULL	Processing completion time
last_error	text	YES	NULL	Most recent failure message
correlation_id	text	YES	NULL	Shared orchestration correlation ID
created_at	timestamptz	NO	now()	Creation timestamp
updated_at	timestamptz	NO	now()	Last update

---

domain_events

Outbox-style domain event log written transactionally with state changes.

Column	Type	Nullable	Default	Description
id	text	NO	uuid	Primary key
event_type	text	NO	-	Event name such as `interview.started`
aggregate_type	text	NO	-	Aggregate category such as `interview`
aggregate_id	text	NO	-	Aggregate identifier
payload_json	text	NO	-	JSON-encoded event payload
correlation_id	text	YES	NULL	Shared orchestration correlation ID
created_at	timestamptz	NO	now()	Creation timestamp

---

interview_scores

Summary scoring for completed interviews.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
interview_id	uuid	NO	-	FK to interviews (UNIQUE)
overall_score	integer	YES	NULL	Weighted score (0-100)
narrative_summary	text	YES	NULL	AI-generated summary
candidate_feedback	text	YES	NULL	Feedback for candidate
anti_cheat_risk_level	text	YES	NULL	low/medium/high
scored_by	text	YES	NULL	'ai' or user ID
model_version	text	YES	NULL	AI model used
prompt_version	text	YES	NULL	Prompt version
rubric_version	text	YES	NULL	Rubric version
human_override	boolean	YES	false	Manual adjustment made
human_override_by	uuid	YES	NULL	Who overrode
human_override_reason	text	YES	NULL	Why overridden
created_at	timestamptz	NO	now()	Creation timestamp
updated_at	timestamptz	NO	now()	Last update

Indexes:

- `interview_scores_pkey` (id)
- `interview_scores_interview_id_idx` (interview_id) UNIQUE

Foreign Keys:

- `interview_scores_interview_id_fkey` - interviews(id)

---

score_dimensions

Individual dimension scores for interviews.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
interview_id	uuid	NO	-	FK to interviews
dimension	text	NO	-	Dimension name
score	numeric	NO	-	Score (0-10)
weight	numeric	YES	NULL	Weight used (0-1)
evidence	text	YES	NULL	Explanation
cited_quotes	jsonb	YES	NULL	Supporting quotes
created_at	timestamptz	NO	now()	Creation timestamp

Dimensions are dynamic and produced by the scoring pipeline. Culture fit dimensions from model-service-1 follow the canonical five (ownership, execution, challenge, ambiguity, feedback). Skills fit dimensions from model-service-2 are role-specific and derived from the job description.

Indexes:

- `score_dimensions_pkey` (id)
- `score_dimensions_interview_id_idx` (interview_id)

Foreign Keys:

- `score_dimensions_interview_id_fkey` - interviews(id)

---

invitations

Interview invitations sent to candidates.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
application_id	uuid	NO	-	FK to applications
token	text	NO	-	Secure invite token
status	invitation_status	NO	'pending'	Invitation status
email_template	text	YES	NULL	Template used
expires_at	timestamptz	NO	-	Expiration time
sent_at	timestamptz	YES	NULL	When email sent
opened_at	timestamptz	YES	NULL	When link clicked
created_at	timestamptz	NO	now()	Creation timestamp

Indexes:

- `invitations_pkey` (id)
- `invitations_token_idx` (token) UNIQUE
- `invitations_application_id_idx` (application_id)

Foreign Keys:

- `invitations_application_id_fkey` - applications(id)

---

practice_interviews

Practice interview sessions (not tied to real jobs).

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
user_id	uuid	NO	-	FK to users
sample_role_type	text	NO	-	Practice role category
status	text	NO	'pending'	Session status
started_at	timestamptz	YES	NULL	Start time
ended_at	timestamptz	YES	NULL	End time
duration_seconds	integer	YES	NULL	Duration
feedback	jsonb	YES	NULL	Practice feedback
created_at	timestamptz	NO	now()	Creation timestamp

---

user_roles

Application-level role assignments.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
user_id	uuid	NO	-	FK to users
role	app_role	NO	-	Role enum value
created_at	timestamptz	NO	now()	When assigned

Indexes:

- `user_roles_pkey` (id)
- `user_roles_user_id_idx` (user_id)

---

audit_log

Audit trail for security and compliance.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
user_id	uuid	YES	NULL	User who took action
organisation_id	uuid	YES	NULL	Related organisation
action	text	NO	-	Action type
entity_type	text	NO	-	What was affected
entity_id	uuid	YES	NULL	ID of affected entity
old_values	jsonb	YES	NULL	Before state
new_values	jsonb	YES	NULL	After state
ip_address	text	YES	NULL	Client IP
created_at	timestamptz	NO	now()	When action occurred

Indexes:

- `audit_log_pkey` (id)
- `audit_log_organisation_id_idx` (organisation_id)
- `audit_log_created_at_idx` (created_at)

Foreign Keys:

- `audit_log_organisation_id_fkey` - organisations(id)

---

data_deletion_requests

GDPR data deletion request tracking.

Column	Type	Nullable	Default	Description
id	uuid	NO	gen_random_uuid()	Primary key
user_id	uuid	NO	-	User requesting deletion
request_type	text	NO	'full_deletion'	Type of deletion
status	text	NO	'pending'	Request status
reason	text	YES	NULL	Why requesting
notes	text	YES	NULL	Processing notes
requested_at	timestamptz	NO	now()	Request time
processed_at	timestamptz	YES	NULL	Completion time
processed_by	text	YES	NULL	Who processed

Request Types:

- `full_deletion` - Delete all data
- `recording_only` - Delete only recordings
- `anonymize` - Anonymize PII, keep aggregate data

Status Values:

- `pending` - Awaiting processing
- `processing` - Currently being processed
- `completed` - Successfully processed
- `failed` - Processing failed

---

org_environment_inputs

Organisation operating-environment survey responses used to configure culture fit scoring.

Column	Type	Nullable	Default	Description
id	text	NO	uuid	Primary key
organisation_id	text	NO	-	FK to organisations (CASCADE)
raw_answers	text	NO	-	JSON-encoded questionnaire answers
signals_json	text	YES	NULL	Extracted behavioural signals
derived_environment	text	NO	-	JSON-encoded derived environment profile
defaulted_variables	text	YES	NULL	Variables that used defaults
extra_fatal_risks	text	YES	NULL	Additional fatal risk flags
submitted_by	text	YES	NULL	User who submitted
created_at	timestamptz	NO	now()	Creation timestamp

Foreign Keys:

- `org_environment_inputs_organisation_id_fkey` - organisations(id) ON DELETE CASCADE

---

resume_ingestion_batches

Bulk resume upload batches created by recruiters for a specific job role.

Column	Type	Nullable	Default	Description
id	text	NO	uuid	Primary key
organisation_id	text	NO	-	FK to organisations
job_role_id	text	NO	-	FK to job_roles
status	text	NO	'draft'	Batch status (draft, processing, completed)
title	text	YES	NULL	Batch label
created_by	text	YES	NULL	FK to users
created_at	timestamptz	NO	now()	Creation timestamp
updated_at	timestamptz	NO	now()	Last update

Foreign Keys:

- `resume_ingestion_batches_organisation_id_fkey` - organisations(id)
- `resume_ingestion_batches_job_role_id_fkey` - job_roles(id)
- `resume_ingestion_batches_created_by_fkey` - users(id)

---

resume_ingestion_items

Individual resume items within a batch, tracking parse status and candidate matching.

Column	Type	Nullable	Default	Description
id	text	NO	uuid	Primary key
batch_id	text	NO	-	FK to resume_ingestion_batches
file_id	text	NO	-	FK to files
parse_status	text	NO	'pending'	Parse status (pending, completed, failed)
recruiter_review_status	text	NO	'pending_review'	Recruiter review state
candidate_email	text	YES	NULL	Extracted or assigned email
candidate_name	text	YES	NULL	Extracted name
parse_confidence_json	text	YES	NULL	JSON parse confidence scores
parse_error	text	YES	NULL	Parse failure message
matched_user_id	text	YES	NULL	FK to users (matched candidate)
candidate_profile_id	text	YES	NULL	FK to candidate_profiles
application_id	text	YES	NULL	FK to applications
snapshot_id	text	YES	NULL	FK to parsed_profile_snapshots
invitation_id	text	YES	NULL	FK to invitations
uploaded_at	timestamptz	YES	NULL	Upload timestamp
processed_at	timestamptz	YES	NULL	Parse completion time
invited_at	timestamptz	YES	NULL	When invitation was sent
created_at	timestamptz	NO	now()	Creation timestamp
updated_at	timestamptz	NO	now()	Last update

Foreign Keys:

- `resume_ingestion_items_batch_id_fkey` - resume_ingestion_batches(id)
- `resume_ingestion_items_file_id_fkey` - files(id)
- `resume_ingestion_items_matched_user_id_fkey` - users(id)
- `resume_ingestion_items_candidate_profile_id_fkey` - candidate_profiles(id)
- `resume_ingestion_items_application_id_fkey` - applications(id)
- `resume_ingestion_items_snapshot_id_fkey` - parsed_profile_snapshots(id)
- `resume_ingestion_items_invitation_id_fkey` - invitations(id)

---

parsed_profile_snapshots

Immutable snapshots of parsed resume data, preserving the extraction result at parse time.

Column	Type	Nullable	Default	Description
id	text	NO	uuid	Primary key
user_id	text	YES	NULL	FK to users (if matched)
file_id	text	NO	-	FK to files (source document)
snapshot_type	text	NO	'resume_parse'	Snapshot category
parser_version	text	YES	NULL	Parser version used
source_kind	text	YES	NULL	Source type (pdf, docx, etc.)
data_json	text	NO	-	JSON-encoded parsed profile data
confidence_json	text	YES	NULL	JSON field-level confidence scores
raw_text	text	YES	NULL	Extracted raw text
created_at	timestamptz	NO	now()	Creation timestamp

Foreign Keys:

- `parsed_profile_snapshots_user_id_fkey` - users(id)
- `parsed_profile_snapshots_file_id_fkey` - files(id)

---

post_hire_outcomes

Post-hire performance data linked to interview scores for predictive validation.

Column	Type	Nullable	Default	Description
id	text	NO	uuid	Primary key
interview_score_id	text	NO	-	FK to interview_scores (CASCADE)
observed_at	timestamptz	NO	-	When performance was observed
snapshot_period	text	NO	'custom'	Observation period (e.g., 30d, 90d, custom)
outcome_rating	float	NO	-	Performance rating
outcome_notes	text	YES	NULL	Freeform notes
dimension_ratings	text	YES	NULL	JSON per-dimension performance ratings
recorded_by	text	YES	NULL	FK to users (SET NULL on delete)
created_at	timestamptz	NO	now()	Creation timestamp

Indexes:

- `ix_post_hire_outcomes_interview_score_id` (interview_score_id)
- `ix_post_hire_outcomes_observed_at` (observed_at)
- `ix_post_hire_outcomes_snapshot_period` (snapshot_period)

Foreign Keys:

- `post_hire_outcomes_interview_score_id_fkey` - interview_scores(id) ON DELETE CASCADE
- `post_hire_outcomes_recorded_by_fkey` - users(id) ON DELETE SET NULL

Enums

app_role

Application-level roles.

```
CREATE TYPE app_role AS ENUM (
  'org_admin',
  'org_recruiter',
  'org_viewer',
  'candidate'
);
```

interview_status

Interview session states.

```
CREATE TYPE interview_status AS ENUM (
  'invited',
  'scheduled',
  'in_progress',
  'completed',
  'cancelled',
  'expired'
);
```

invitation_status

Invitation states.

```
CREATE TYPE invitation_status AS ENUM (
  'pending',
  'sent',
  'delivered',
  'opened',
  'bounced',
  'expired'
);
```

job_role_status

Job listing states.

```
CREATE TYPE job_role_status AS ENUM (
  'draft',
  'active',
  'paused',
  'closed'
);
```

Functions

user_belongs_to_org

Check if user is member of organisation.

```
CREATE FUNCTION user_belongs_to_org(_org_id uuid, _user_id uuid)
RETURNS boolean AS $$
  SELECT EXISTS (
    SELECT 1 FROM org_users
    WHERE organisation_id = _org_id AND user_id = _user_id
  );
$$ LANGUAGE sql SECURITY DEFINER;
```

user_org_role

Get user's role in organisation.

```
CREATE FUNCTION user_org_role(_org_id uuid, _user_id uuid)
RETURNS text AS $$
  SELECT role FROM org_users
  WHERE organisation_id = _org_id AND user_id = _user_id;
$$ LANGUAGE sql SECURITY DEFINER;
```

get_user_org_id

Get user's primary organisation.

```
CREATE FUNCTION get_user_org_id(_user_id uuid)
RETURNS uuid AS $$
  SELECT organisation_id FROM org_users
  WHERE user_id = _user_id
  LIMIT 1;
$$ LANGUAGE sql SECURITY DEFINER;
```

has_role

Check if user has specific app role.

```
CREATE FUNCTION has_role(_role app_role, _user_id uuid)
RETURNS boolean AS $$
  SELECT EXISTS (
    SELECT 1 FROM user_roles
    WHERE user_id = _user_id AND role = _role
  );
$$ LANGUAGE sql SECURITY DEFINER;
```

Access Control Policy Summary

> **Important:** The policies below describe the *intended* access control matrix. These are currently enforced at the **application layer** (FastAPI dependency injection in `backend/app/api/deps.py`) and **not** via PostgreSQL Row-Level Security (RLS). No RLS policies exist in the Alembic migrations. See ARCHITECTURE_OVERVIEW.md, Appendix C, IMP-03 for the related audit finding.

Table	SELECT	INSERT	UPDATE	DELETE
organisations	Own org members	Service role	Org admins	-
org_users	Own org members	Service role	Org admins	Org admins
job_roles	Org members + public active	Org recruiters	Org recruiters	Org admins
candidate_profiles	Own + org with application	Own only	Own only	Own only
candidate_skills	Own + org with application	Own only	Own only	Own only
education	Own + org with application	Own only	Own only	Own only
employment_history	Own + org with application	Own only	Own only	Own only
candidate_dei	Own only	Own only	Own only	Own only
applications	Own + org members	Candidates	Org members	-
interviews	Own + org members	System	System	-
transcript_segments	Own + org members	System	-	-
interview_scores	Own + org members	System	Org admins	-
score_dimensions	Own + org members	System	-	-
invitations	Org members	Org recruiters	Org recruiters	-
practice_interviews	Own only	Own only	Own only	Own only
audit_log	Org admins	System	-	-
data_deletion_requests	Own only	Own only	System	-

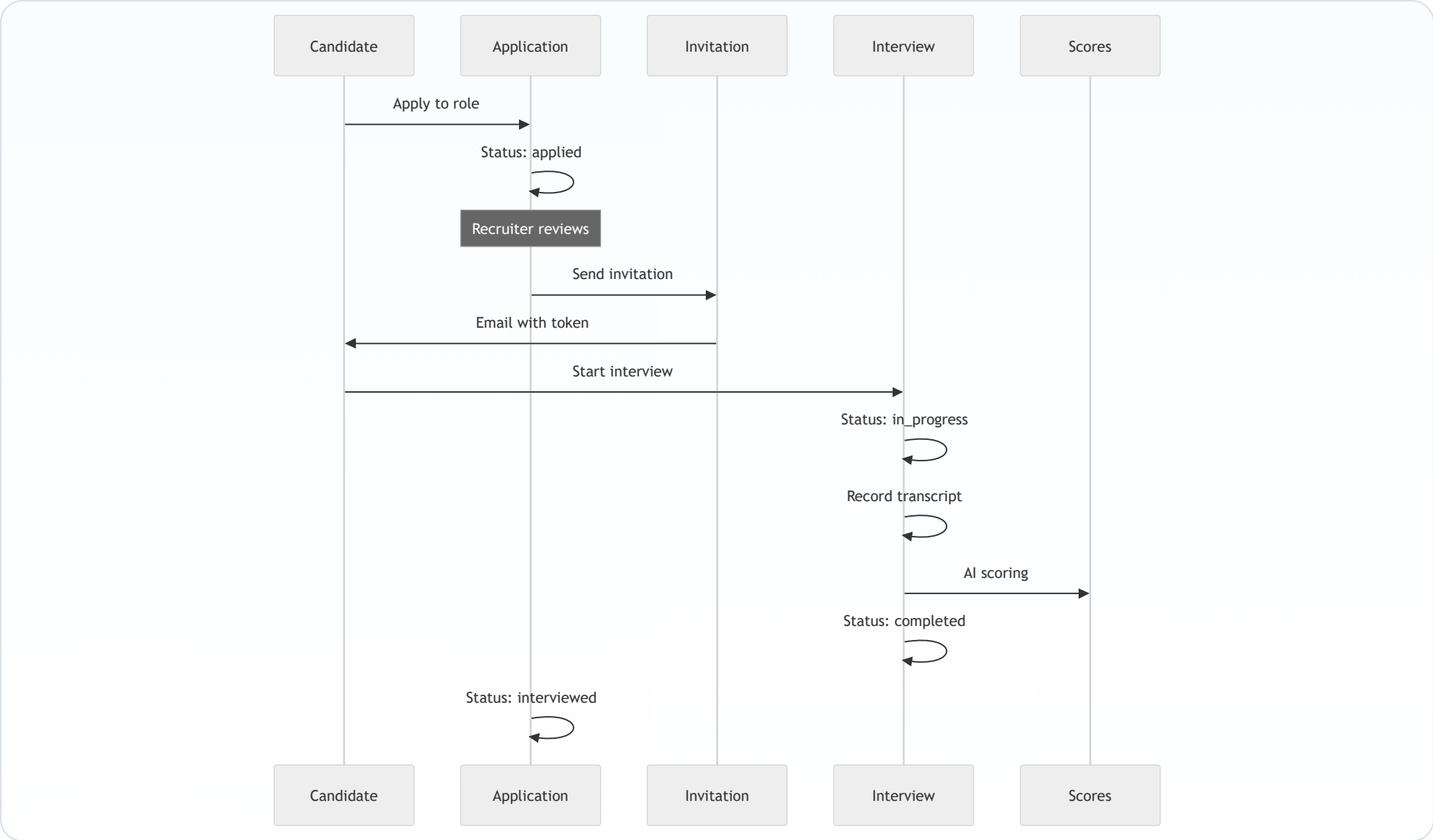
Data Flow Patterns

Candidate Application Flow

DATABASE SCHEMA

Candidate Application Flow

documentation/DATABASE_SCHEMA.md

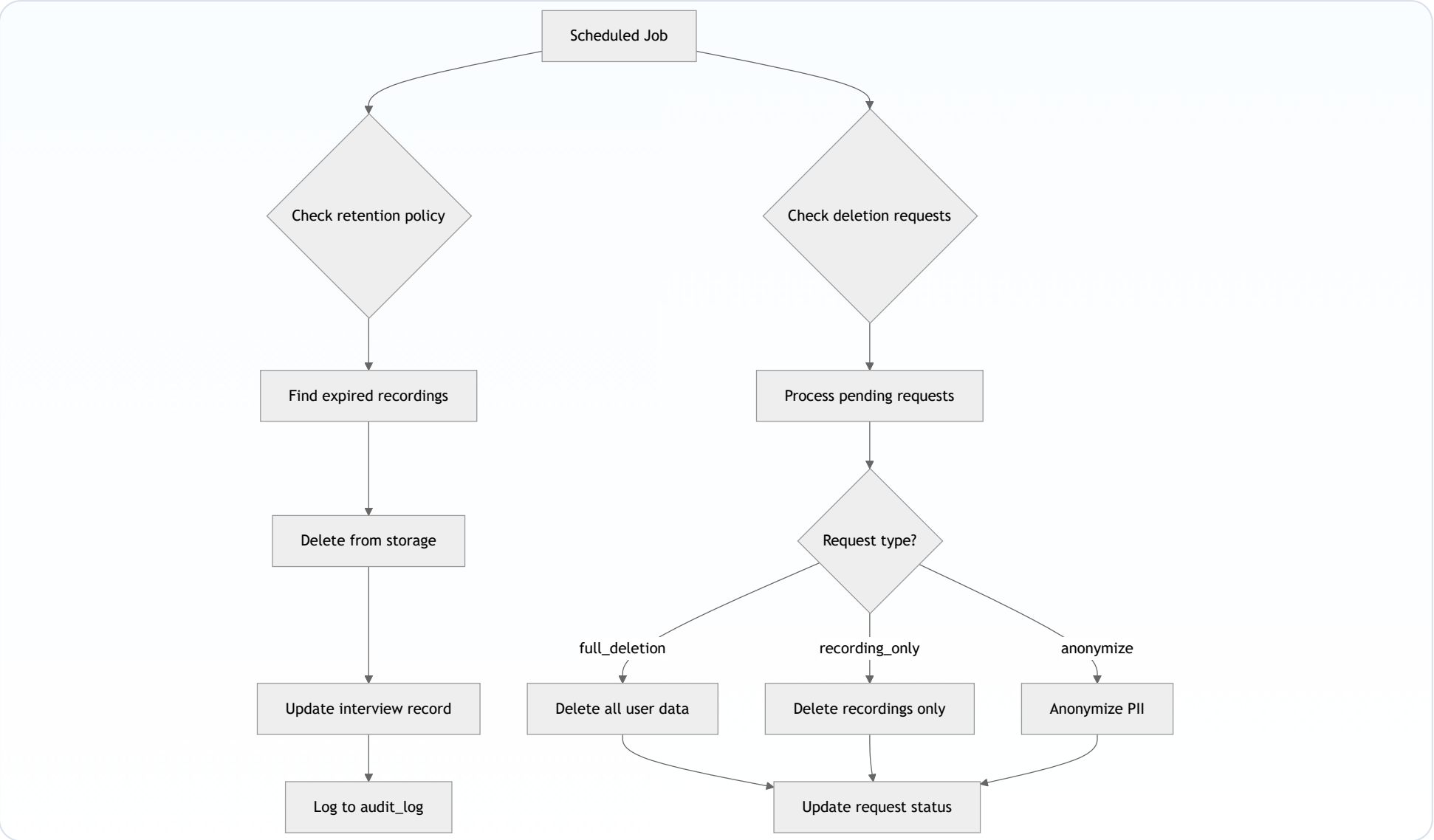


Data Retention Flow

DATABASE SCHEMA

Data Retention Flow

documentation/DATABASE_SCHEMA.md



COMPILED SOURCE DOCUMENT

Architecture Decisions

`documentation/ARCHITECTURE_DECISIONS.md`

ADR history preserving the rationale behind the core structural choices reflected in the implemented platform.

> Last Updated: April 2026
>
> **Note:** The canonical ADR reference with full evidence tracing is now in ARCHITECTURE_OVERVIEW.md, Section 19. This file is retained as the original ADR source and remains accurate.

This document captures the current architecture for Talenti. The system is a FastAPI + PostgreSQL backend with a React frontend, two ML model scoring services, an Azure Communication Services worker, and Azure service integrations.

ADR-001: FastAPI + PostgreSQL Backend

Status: Accepted

Decision: Use FastAPI for the backend API layer and PostgreSQL as the primary datastore.

Rationale:

- Simple, lightweight local development experience.
- Consistent with current `DATABASE_URL` defaults and migrations.
- No external database dependency required for initial deployments.

ADR-002: JWT Authentication

Status: Accepted

Decision: Use JWTs issued and validated by the FastAPI service with a local user table.

Rationale:

- Keeps auth in the same service boundary as the API.
- Enables local development without third-party auth providers.

ADR-003: Azure Integrations

Status: Accepted

Decision: Use Azure Communication Services, Azure Speech, Azure OpenAI, and Azure Blob Storage for communications, transcription, AI responses, and file storage.

Rationale:

- Provides scalable managed services aligned with product requirements.
- Keeps the backend responsible for access and key management.

ADR-004: Frontend + API Contract

Status: Accepted

Decision: The React frontend communicates with the FastAPI backend via REST endpoints under `/api` and `/api/v1` prefixes.

Rationale:

- Clear versioning for API endpoints.
- Compatible with existing frontend API clients.

ADR-005: Database-Backed Background Jobs

Status: Accepted

Decision: Use a `background_jobs` table polled by `backend-worker` instead of an external message broker (e.g., RabbitMQ, Azure Service Bus).

Rationale:

- Avoids introducing an external broker dependency for the current workload scale.
- Jobs are transactionally consistent with the state changes that produce them.
- The `domain_events` outbox pattern ensures events are not lost even if the worker is temporarily unavailable.
- Retry logic with exponential backoff and max attempts is built into the worker.

ADR-006: Dual Scorecard Architecture

Status: Accepted

Decision: Interview scoring is split into two independent model services: model-service-1 (culture/behavioural fit) and model-service-2 (skills/technical fit). Scores are never merged into a single composite number.

Rationale:

- Culture fit and skills fit are orthogonal assessments that should be evaluated independently.
- Keeping them separate allows recruiters to weight and interpret each dimension on its own terms.
- Each model service can evolve, be retrained, or be replaced independently without affecting the other.
- The five canonical culture dimensions (ownership, execution, challenge, ambiguity, feedback) are structurally different from the JD-driven skills dimensions.

ADR-007: Decision-Dominant Scoring Logic

Status: Accepted

Decision: ML classifiers extract signals from interview transcripts, but all scoring decisions are made by deterministic rules with configurable thresholds.

Rationale:

- Deterministic decisions are auditable and legally defensible — important for hiring decisions.
- Scoring rules can be explained to candidates and recruiters without exposing ML internals.
- Confidence gating ensures the system flags uncertainty rather than producing overconfident scores.
- Fatal risk flags can override all other signals, providing a safety mechanism.

ADR-008: Python ACS Service Separation

Status: Accepted

Decision: Azure Communication Services call automation runs as a separate `python-acs-service` microservice with internal-only ingress, not as part of the main backend.

Rationale:

- Call automation has different scaling characteristics and failure modes from the main API.
- Isolating ACS logic prevents call-related issues from affecting core API availability.
- The service communicates with the backend via shared-secret-authenticated callbacks.
- Internal-only ingress reduces the attack surface.

ADR-009: Trunk-Based Release with Immutable Promotion

Status: Accepted

Decision: Use trunk-based development on `main` with release-please for versioning. Container images are built once on `main`, signed, attested, and promoted immutably through DEV -> UAT -> PROD without rebuilding.

Rationale:

- Build-once-promote-many eliminates "works on DEV but not PROD" class of issues.
- Image signing and attestation provide supply-chain integrity verification at each promotion gate.
- `release-manifest.json` captures exact image digests for reproducible deployments.
- UAT auto-promotes from published releases; PROD requires manual approval by release tag.

COMPILED SOURCE DOCUMENT

Frontend Architecture Guide

`documentation/FRONTEND_GUIDE.md`

Frontend implementation and browser-side architecture reference covering routing, state patterns, forms, API integration, and UI structure.

> **Version:** 1.1.0
> **Last Updated:** April 2026
> **Stack:** React 18 + JavaScript (JSX) + Vite + Tailwind CSS

Overview

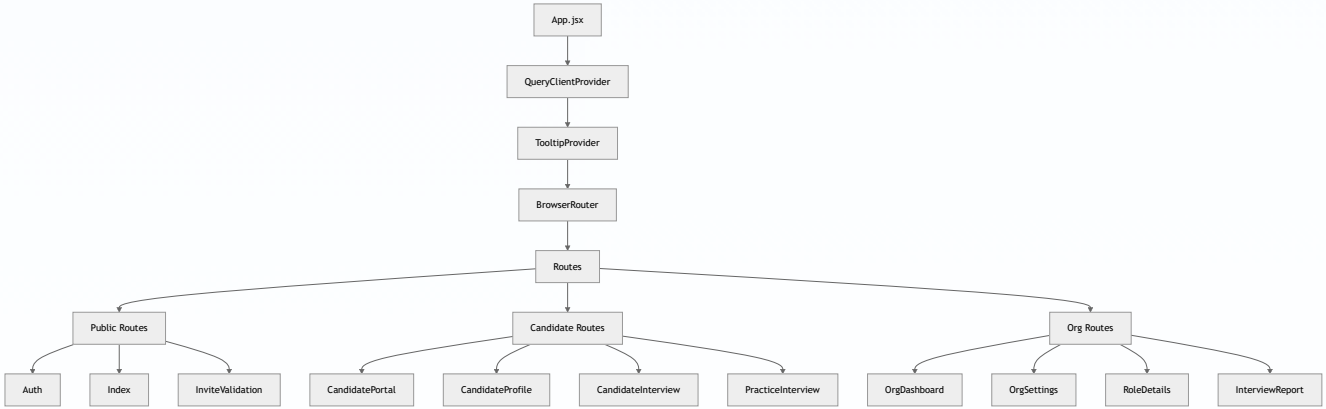
This guide documents the frontend architecture, component patterns, state management, and development conventions for the Talenti AI Interview Platform.

Architecture Overview

FRONTEND ARCHITECTURE GUIDE

Architecture Overview

documentation/FRONTEND_GUIDE.md



Technology Stack

Layer	Technology
Framework	React 18
Language	JavaScript (JSX)
Build	Vite
Styling	Tailwind CSS + shadcn/ui
State	React Query (TanStack Query)
Forms	React Hook Form + Zod
Routing	React Router v6
Backend	FastAPI + PostgreSQL

Project Structure

```
src/
├── main.jsx                        # Entry point
├── api/                            # API clients for FastAPI backend
│   ├── http.js                     # HTTP client with JWT token handling
│   ├── auth.js                     # Authentication (register, login, claim-invite)
│   ├── candidates.js               # Candidate profiles, employment, education, skills
│   ├── interviews.js               # Interview lifecycle, transcripts, scoring
│   ├── invitations.js              # Interview invitations
│   ├── organisations.js            # Organisation CRUD, stats, retention
│   ├── roles.js                    # Job role CRUD, applications
│   ├── resumeBatches.js            # Bulk resume upload and processing
│   ├── acs.js                      # Azure Communication Services
│   ├── audit.js                    # Audit trail
│   ├── requirements.js             # Role requirements extraction
│   ├── resume.js                   # Resume handling
│   ├── scoring.js                  # Interview scoring
│   ├── shortlist.js                # Shortlist management
│   ├── speech.js                   # Speech services
│   └── storage.js                  # Azure Blob Storage upload URLs
│
└── lib/
    ├── App.jsx                     # Root component with routing
    ├── index.css                   # Tailwind + design tokens
    │
    ├── components/
    │   ├── ui/                     # shadcn/ui primitives (button, card, dialog, etc.)
    │   ├── AuditTrailViewer.jsx
    │   ├── AvatarRenderer.jsx      # Azure Avatar AI interviewer rendering
    │   ├── CallControls.jsx        # Video call mute/video/hangup controls
    │   ├── CandidateComparison.jsx
    │   ├── DataRetentionSettings.jsx
    │   ├── ProfileManagement.jsx
    │   ├── ResumeBatchUploadDialog.jsx  # Bulk resume upload for recruiters
    │   ├── SendInvitationDialog.jsx
    │   ├── ShortlistView.jsx
    │   └── VideoRenderer.jsx       # Local/remote video stream rendering
    │
    ├── hooks/
    │   ├── useAcsCall.js           # Azure Communication Services call management
    │   ├── useAcsToken.js          # ACS token generation
    │   ├── useAuditLog.js          # Audit logging
    │   ├── useAzureAvatar.js       # Azure Avatar lifecycle and speaking
    │   ├── useAzureSpeech.js       # Speech services (TTS/STT)
    │   ├── useCandidateData.js     # Candidate profile, applications, invitations
    │   ├── useDeletionRequests.js
    │   ├── useInterviewContext.js   # Context-Aware Generation for competencies
    │   ├── useInterviewPersistence.js
    │   ├── useInvitations.js
    │   ├── useOrgData.js           # Organisation, roles, and stats
    │   ├── useShortlist.js
    │   ├── useSpeechRecognition.js
    │   ├── useSpeechSynthesis.js
    │   └── use-toast.js
```

```
    │
    ├── lib/
    │   ├── auditLog.js
    │   ├── generateInterviewReport.js
    │   ├── scoring.js
    │   └── utils.js                # cn(), formatters
    │
    └── pages/
        ├── Auth.jsx
        ├── CandidateInterview.jsx
        ├── CandidatePortal.jsx
        ├── CandidateProfile.jsx
        ├── EditRoleRubric.jsx
        ├── Index.jsx
        ├── InterviewComplete.jsx
        ├── InterviewLobby.jsx
        ├── InterviewReport.jsx
        ├── InviteValidation.jsx
        ├── LiveInterview.jsx
        ├── NewRole.jsx
        ├── NotFound.jsx
        ├── OrgDashboard.jsx
        ├── OrgOnboarding.jsx
        ├── OrgSettings.jsx
        ├── PracticeInterview.jsx
        ├── PracticeInterviewComplete.jsx
        └── RoleDetails.jsx
```

Component Patterns

Component File Structure

```
// ComponentName.jsx
import { useState, useEffect } from "react";
import { useQuery } from "@tanstack/react-query";
import { Button } from "@/components/ui/button";
import { candidatesApi } from "@/api/candidates";

export function ComponentName({ userId, onComplete }) {
  const [isLoading, setIsLoading] = useState(false);

  const { data, isLoading: queryLoading } = useQuery({
    queryKey: ["data", userId],
    queryFn: () => candidatesApi.getProfile(userId),
  });

  const handleSubmit = async () => {
    // ...
  };

  return (
    <div className="container mx-auto p-4">
      {/* JSX */}
    </div>
  );
}
```

Container vs Presentational Components

```
// Container: handles data/logic
function CandidateProfileContainer() {
  const { profile, isLoading, updateProfile } = useCandidateData();

  if (isLoading) return <Skeleton />;

  return <ProfileForm profile={profile} onUpdate={updateProfile} />;
}

// Presentational: handles UI
function ProfileForm({ profile, onUpdate }) {
  return (
    <form onSubmit={handleSubmit}>
      {/* Pure UI, no data fetching */}
    </form>
  );
}
```

shadcn/ui Component Customization

```
// src/lib/components/ui/button.jsx
import { cva } from "class-variance-authority";

const buttonVariants = cva(
  // Base styles
  "inline-flex items-center justify-center rounded-md text-sm font-medium transition-colors",
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground hover:bg-primary/90",
        destructive: "bg-destructive text-destructive-foreground hover:bg-destructive/90",
        outline: "border border-input bg-background hover:bg-accent hover:text-accent-foregrour
        secondary: "bg-secondary text-secondary-foreground hover:bg-secondary/80",
        ghost: "hover:bg-accent hover:text-accent-foreground",
        link: "text-primary underline-offset-4 hover:underline",
        // Custom variants for Talenti
        interview: "bg-primary text-primary-foreground shadow-lg hover:shadow-xl",
      },
      size: {
        default: "h-10 px-4 py-2",
        sm: "h-9 rounded-md px-3",
        lg: "h-11 rounded-md px-8",
        icon: "h-10 w-10",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
);
```

State Management

React Query for Server State

```
// Query - fetching data
const { data, isLoading, error, refetch } = useQuery({
  queryKey: ["interviews", roleId],
  queryFn: () => interviewsApi.listRoleApplications(roleId),
  staleTime: 5 * 60 * 1000, // 5 minutes
});

// Mutation - modifying data
const { mutate, isPending } = useMutation({
  mutationFn: (payload) => interviewsApi.create(payload),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ["interviews"] });
    toast.success("Interview created");
  },
  onError: (error) => {
    toast.error(error.message);
  },
});
```

Local State Patterns

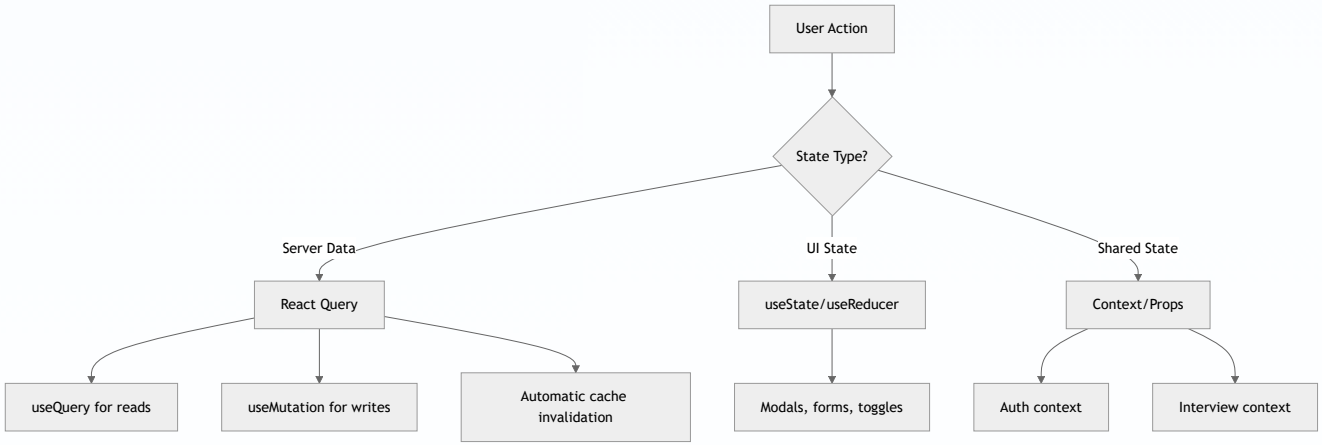
```
// Simple state
const [isOpen, setIsOpen] = useState(false);

// Complex state with reducer
const [state, dispatch] = useReducer(interviewReducer, initialState);

// Derived state
const completedInterviews = useMemo(
  () => interviews.filter(i => i.status === 'completed'),
  [interviews]
);
```

State Flow Diagram

FRONTEND ARCHITECTURE GUIDE

State Flow Diagram

documentation/FRONTEND_GUIDE.md



Routing

Route Structure

```
// App.jsx
<Routes>
  {/* Public routes */}
  <Route path="/" element={<Index />} />
  <Route path="/auth" element={<Auth />} />
  <Route path="/invite/:token" element={<InviteValidation />} />

  {/* Organisation routes */}
  <Route path="/org" element={<OrgDashboard />} />
  <Route path="/org/onboarding" element={<OrgOnboarding />} />
  <Route path="/org/settings" element={<OrgSettings />} />
  <Route path="/org/new-role" element={<NewRole />} />
  <Route path="/org/role/:roleId" element={<RoleDetails />} />
  <Route path="/org/role/:roleId/rubric" element={<EditRoleRubric />} />
  <Route path="/org/interview/:interviewId" element={<InterviewReport />} />

  {/* Candidate routes */}
  <Route path="/candidate/portal" element={<CandidatePortal />} />
  <Route path="/candidate/profile" element={<CandidateProfile />} />
  <Route path="/candidate/practice" element={<PracticeInterview />} />
  <Route path="/candidate/practice/complete" element={<PracticeInterviewComplete />} />
  <Route path="/candidate/:inviteId/lobby" element={<InterviewLobby />} />
  <Route path="/candidate/:inviteId/live" element={<LiveInterview />} />
  <Route path="/candidate/:inviteId" element={<CandidateInterview />} />
  <Route path="/candidate/complete" element={<InterviewComplete />} />

  {/* 404 */}
  <Route path="*" element={<NotFound />} />
</Routes>
```

Protected Route Pattern

```
function ProtectedRoute({ children, requiredRole }) {
  const { user, isLoading } = useAuth();
  const navigate = useNavigate();

  useEffect(() => {
    if (!isLoading && !user) {
      navigate('/auth', { replace: true });
    }
  }, [user, isLoading, navigate]);

  if (isLoading) return <LoadingSpinner />;
  if (!user) return null;

  return children;
}

// Usage
<Route
  path="/org"
  element={
    <ProtectedRoute requiredRole="org_admin">
      <OrgDashboard />
    </ProtectedRoute>
  }
/>
```

Navigation Helpers

```
// Programmatic navigation
const navigate = useNavigate();
navigate('/org');
navigate(-1); // Go back

// With state
navigate('/candidate/complete', {
  state: { interviewId, score }
});

// Access state
const location = useLocation();
const { interviewId } = location.state || {};
```

Form Handling

React Hook Form + Zod Pattern

```
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';

// 1. Define schema
const profileSchema = z.object({
  firstName: z.string().min(1, 'Required').max(100),
  lastName: z.string().min(1, 'Required').max(100),
  email: z.string().email('Invalid email'),
  phone: z.string().regex(/^\+?[0-9\s-]{8,20}$/, 'Invalid phone').optional(),
  availability: z.enum(['immediate', '2_weeks', '1_month', 'flexible']),
});

// 2. Create form
function ProfileForm() {
  const form = useForm({
    resolver: zodResolver(profileSchema),
    defaultValues: {
      firstName: '',
      lastName: '',
      email: '',
      availability: 'flexible',
    },
  });

  const onSubmit = async (data) => {
    await updateProfile(data);
  };

  return (
    <Form {...form}>
      <form onSubmit={form.handleSubmit(onSubmit)}>
        <FormField
          control={form.control}
          name="firstName"
          render={({ field }) => (
            <FormItem>
              <FormLabel>First Name</FormLabel>
              <FormControl>
                <Input {...field} />
              </FormControl>
              <FormMessage />
            </FormItem>
          )}
        />
        {/* More fields... */}
        <Button type="submit" disabled={form.formState.isSubmitting}>
          Save
        </Button>
      </form>
    </Form>
```

```
  );
}
```

Form with File Upload

```
const { register, handleSubmit } = useForm();

const onSubmit = async (data) => {
  const file = data.cv[0];
  const { upload_url } = await storageApi.createUploadUrl({
    file_name: file.name,
    content_type: file.type,
  });
  await fetch(upload_url, { method: "PUT", body: file });
};

<input type="file" {...register("cv")} accept=".pdf,.doc,.docx" />
```

Styling Conventions

Design Token Usage

```
// ✅ CORRECT - Use semantic tokens
<div className="bg-background text-foreground">
  <Button className="bg-primary text-primary-foreground">
    Submit
  </Button>
</div>

// ❌ WRONG - Direct colors
<div className="bg-white text-black">
  <Button className="bg-orange-500 text-white">
    Submit
  </Button>
</div>
```

Available Design Tokens

```
/* Colors (from index.css) */
--background       /* Main background */
--foreground       /* Main text */
--card             /* Card backgrounds */
--card-foreground  /* Card text */
--primary          /* Primary actions (orange) */
--primary-foreground
--secondary        /* Secondary actions */
--secondary-foreground
--muted            /* Muted/disabled */
--muted-foreground
--accent           /* Accent elements */
--accent-foreground
--destructive      /* Error/delete actions */
--destructive-foreground
--border           /* Borders */
--input            /* Input borders */
--ring             /* Focus rings */
```

Responsive Design

```
// Mobile-first approach
<div className="
  p-4           // Mobile: 16px padding
  md:p-6        // Tablet: 24px padding
  lg:p-8        // Desktop: 32px padding

  grid
  grid-cols-1   // Mobile: 1 column
  md:grid-cols-2 // Tablet: 2 columns
  lg:grid-cols-3 // Desktop: 3 columns
  gap-4
">
```

Dark Mode Support

```
// Automatic via CSS variables
<div className="bg-background text-foreground">
  {/* Works in both light and dark mode */}
</div>

// Explicit dark mode styles (rarely needed)
<div className="bg-card dark:bg-card">
  <p className="text-foreground dark:text-foreground">
    Text adjusts automatically
  </p>
</div>
```

Hooks Library

useCandidateData

```
const {
  profile,           // Current user's profile
  isLoading,         // Loading state
  updateProfile,     // Mutation function
  uploadCV,          // Upload resume
  parseResume,       // AI parse resume
  skills,            // Candidate skills array
  education,         // Education history
  employment,        // Employment history
} = useCandidateData();
```

useOrgData

```
const {
  organisation,      // Current org details
  roles,             // Job roles array
  members,           // Org team members
  isAdmin,           // Is current user admin
  createRole,        // Create job role
  updateOrg,         // Update org settings
} = useOrgData();
```

useInterviewContext

```
const {
  interview,         // Current interview
  transcript,        // Live transcript segments
  currentQuestion,   // Current question index
  isRecording,       // Recording state
  startInterview,    // Begin interview
  endInterview,      // End and score
  addTranscript,     // Add transcript segment
} = useInterviewContext(interviewId);
```

useAzureSpeech

```
const {
  isListening,       // STT active
  transcript,        // Current STT result
  startListening,    // Begin STT
  stopListening,     // Stop STT
  speak,             // TTS function
  isSpeaking,        // TTS active
} = useAzureSpeech();
```

API Integration

FastAPI Client Usage

```
// Select
const roles = await rolesApi.listAll({ organisation_id: orgId });

// Insert
const application = await candidatesApi.createApplication({
  candidate_profile_id: profileId,
  job_role_id: roleId,
});

// Update
await candidatesApi.updateProfile(userId, { first_name: "John" });

// Delete
await candidatesApi.deleteSkill(skillId);
```

AI Interview Calls

```
const response = await interviewsApi.aiInterviewer({
  interview_id,
  messages,
  job_title,
  job_description,
});
```

Performance Patterns

Code Splitting

```
// Lazy load heavy components
const InterviewReport = lazy(() => import('@/pages/InterviewReport'));
const VideoRenderer = lazy(() => import('@/components/VideoRenderer'));

// Usage with Suspense
<Suspense fallback={<LoadingSpinner />}>
  <InterviewReport />
</Suspense>
```

Memoization

```
// Expensive computations
const sortedCandidates = useMemo(
  () => candidates.sort((a, b) => b.matchScore - a.matchScore),
  [candidates]
);

// Callback stability
const handleSubmit = useCallback(async () => {
  await saveInterview(data);
}, [data]);

// Component memoization
const CandidateCard = memo(function CandidateCard({ candidate }) {
  return <div>{/* ... */}</div>;
});
```

Query Optimization

```
// Prefetch on hover
const queryClient = useQueryClient();

const prefetchCandidate = (id) => {
  queryClient.prefetchQuery({
    queryKey: ['candidate', id],
    queryFn: () => fetchCandidate(id),
  });
};

<Link
  to={`/candidate/${id}`}
  onMouseEnter={() => prefetchCandidate(id)}
>

// Stale time to reduce refetches
useQuery({
  queryKey: ['roles'],
  queryFn: fetchRoles,
  staleTime: 5 * 60 * 1000, // 5 minutes
  gcTime: 30 * 60 * 1000,   // 30 minutes cache
});
```

Image Optimization

```
// Lazy loading images
<img
  src={avatarUrl}
  loading="lazy"
  alt="Profile"
/>

// Placeholder while loading
<Avatar>
  <AvatarImage src={avatarUrl} />
  <AvatarFallback>{initials}</AvatarFallback>
</Avatar>
```

Testing (Recommended Patterns)

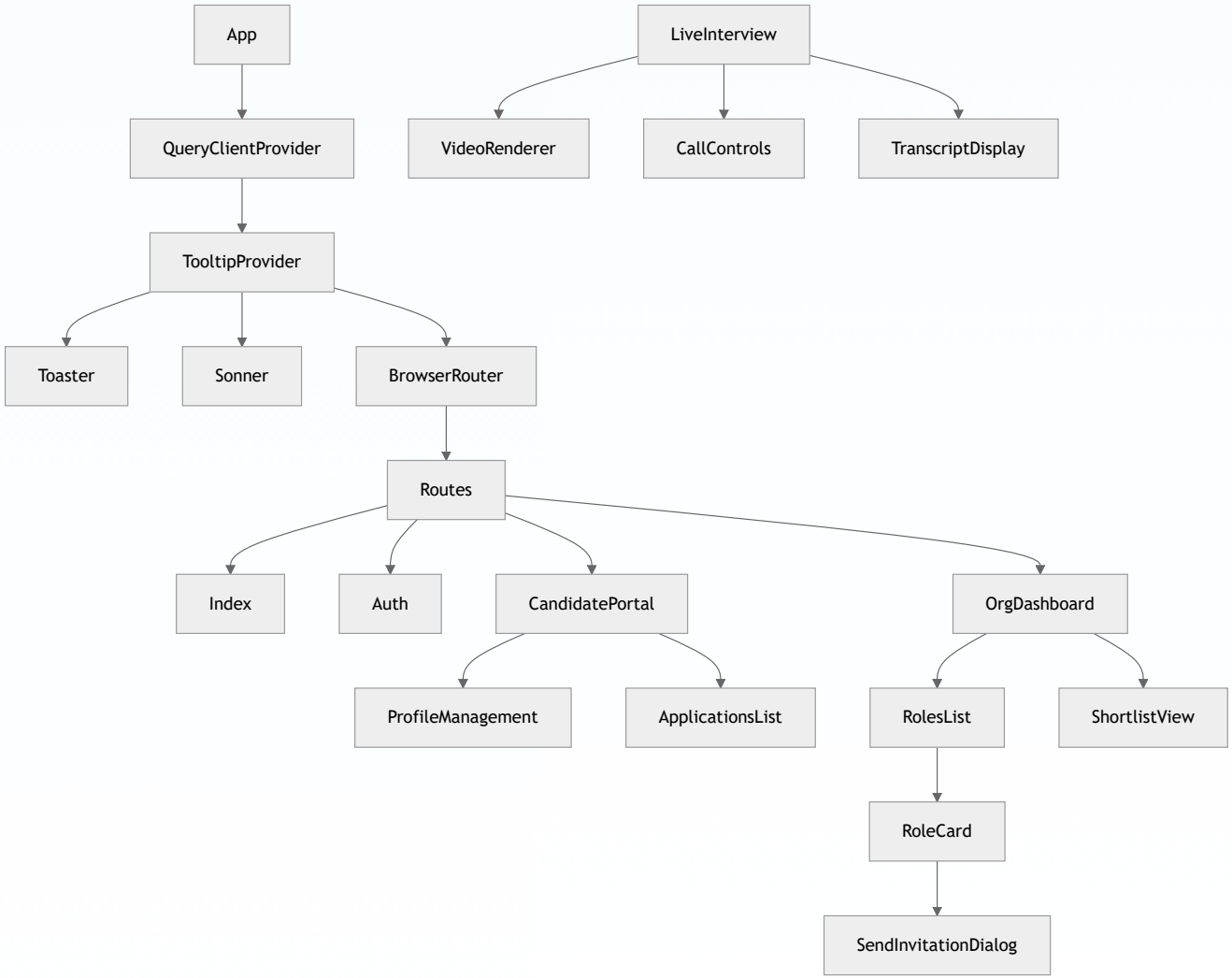
Component Testing

```
import { render, screen, fireEvent } from '@testing-library/react';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

const queryClient = new QueryClient();

function renderWithProviders(component) {
  return render(
    <QueryClientProvider client={queryClient}>
      {component}
    </QueryClientProvider>
  );
}

test('profile form submits correctly', async () => {
  renderWithProviders(<ProfileForm />);

  fireEvent.change(screen.getByLabelText('First Name'), {
    target: { value: 'John' },
  });

  fireEvent.click(screen.getByRole('button', { name: /save/i }));

  expect(await screen.findByText(/saved/i)).toBeInTheDocument();
});
```

Appendix: Component Hierarchy

Appendix: Component Hierarchy

documentation/FRONTEND_GUIDE.md



COMPILED SOURCE DOCUMENT

User and Operational Workflows

`documentation/USER_GUIDE.md`

User-facing flows for candidates, recruiters, and administrators, including audit log access, reporting, privacy handling, and support-oriented operations.

> **Version:** 1.1.0
> **Last Updated:** April 2026

Welcome to Talenti, the AI-powered interview platform. This guide covers how to use Talenti for different user roles.

Getting Started

Creating an Account

1. Visit the Talenti homepage
2. Click **Sign Up**
3. Choose your account type:
    - **Candidate**: Looking for jobs
    - **Organisation**: Hiring talent
4. Enter your email and create a password
5. Verify your email (if required)

Accepting an Interview Invitation (Claim Flow)

If a recruiter has uploaded your resume and invited you directly:

1. Check your email for the invitation link
2. Click the link — you'll be taken to the invitation validation page
3. Create your account using the email from the invitation
4. Your profile will be pre-filled from the parsed resume
5. Review and confirm your details
6. Proceed to the interview lobby

Logging In

1. Click **Sign In**
2. Enter your email and password
3. You'll be directed to your dashboard

For Candidates

Setting Up Your Profile

A complete profile helps employers understand your qualifications.

Personal Information

1. Navigate to **Profile**
2. Fill in your details:
    - Name
    - Email
    - Phone number
    - Location
    - LinkedIn profile (optional)
    - Portfolio URL (optional)

Uploading Your CV/Resume

1. Click **Upload CV**
2. Select a PDF or Word document
3. Talenti will automatically extract:
    - Work experience
    - Education
    - Skills

Tip: Review the extracted information and make corrections if needed.

Adding Skills

1. Go to **Skills** section
2. Click **Add Skill**
3. Enter skill name (e.g., "Python", "Project Management")
4. Select proficiency level

Work Experience

1. Go to **Experience** section
2. Click **Add Experience**
3. Fill in:
    - Company name
    - Job title
    - Start/end dates
    - Description of responsibilities

Practice Interviews

Before your real interview, practice with our AI interviewer.

1. From your dashboard, click **Practice Interview**
2. Select a practice category:
    - General
    - Technical
    - Behavioural
3. Click **Start Practice**
4. Allow microphone and camera access
5. The AI will ask you questions
6. Speak clearly into your microphone
7. When finished, you'll receive feedback

Tips for Practice:

- Find a quiet, well-lit space
- Test your audio before starting
- Practice answering out loud
- Review feedback to improve

Taking a Real Interview

When you're invited to interview:

1. Check your email for the invitation
2. Click the **Start Interview** link
3. You'll see the interview lobby with:
    - Company name
    - Role title
    - Interview duration estimate

Before Starting:

- [ ] Find a quiet location
- [ ] Ensure stable internet connection
- [ ] Test your camera and microphone
- [ ] Close other applications
- [ ] Have water nearby

During the Interview:

1. Click **Begin Interview**
2. The AI interviewer will introduce itself
3. Answer each question clearly
4. Take your time to think before answering
5. The AI will ask follow-up questions
6. After 5-6 questions, the interview will conclude

After the Interview:

1. You'll see a completion confirmation
2. The recording is processed (if applicable)
3. Results are shared with the employer
4. You may receive feedback

Managing Privacy

Control what employers can see:

1. Go to **Settings** → **Privacy**
2. Choose visibility options:
    - **Full Profile**: Employers see everything
    - **Limited**: Only basic info shown
    - **Hidden**: Pause your job search

Requesting Data Deletion

Under GDPR, you can request deletion of your data:

1. Go to **Settings** → **Data & Privacy**
2. Click **Request Deletion**
3. Choose deletion type:
    - Delete everything
    - Delete recordings only
    - Anonymize data
4. Confirm your request

For Recruiters

Organisation Setup

If you're setting up a new organisation:

1. Complete onboarding after signup
2. Enter company details:
    - Company name
    - Industry
    - Website
    - Description
3. Add company values (used for culture fit scoring)

Creating Job Roles

1. Click **New Role** from your dashboard

2. Fill in role details:

    - Title
    - Department
    - Location
    - Work type (remote/hybrid/onsite)
    - Employment type
    - Salary range (optional)

3. Add job description

4. Click **Extract Requirements** to have AI identify:

    - Required skills
    - Experience requirements
    - Qualifications
    - Suggested interview questions

5. Review and edit as needed

6. Set role status:

    - **Draft**: Not visible
    - **Active**: Accepting applications
    - **Paused**: Temporarily hidden
    - **Closed**: No longer accepting

Customizing Scoring Rubric

Adjust how candidates are evaluated:

1. Open the role → **Edit Rubric**
2. Modify dimension weights and thresholds
3. Save changes

Talenti produces a **dual scorecard** for each interview:

- **Culture/Behavioural Fit** — scored across five dimensions: ownership, execution, challenge, ambiguity, and feedback. Based on your organisation's operating environment questionnaire.
- **Skills/Technical Fit** — scored against the role's specific requirements extracted from the job description. Each competency receives a PASS / REVIEW / FAIL outcome.

These two scorecards are presented independently and are never merged into a single composite score.

Inviting Candidates

When you're ready to interview a candidate:

1. Open the candidate's application
2. Click **Send Interview Invitation**
3. Confirm email address
4. Set expiration (default 7 days)
5. Click **Send**

The candidate will receive an email with a unique interview link.

Bulk Resume Upload

Upload multiple resumes at once to quickly onboard candidates for a role:

1. Open a job role
2. Click **Upload Resumes**
3. Select multiple PDF, DOCX, or TXT files
4. Talenti parses each resume automatically, extracting name, email, skills, and experience
5. Review parsed results — confirm or edit candidate emails
6. Click **Invite All** to send interview invitations to the batch
7. Candidates receive invitation emails and follow the claim flow to create their accounts

Generating Shortlists

Use AI to rank candidates:

1. From the role page, click **Generate Shortlist**
2. AI analyzes all applicants against requirements
3. Candidates are scored 0-100
4. View match reasons for each candidate
5. Compare candidates side-by-side

Reviewing Interviews

After candidates complete interviews:

1. Go to **Interviews** tab
2. Click on a completed interview
3. View:
    - **Culture Fit Score** — overall alignment and per-dimension breakdown (ownership, execution, challenge, ambiguity, feedback)
    - **Skills Fit Score** — per-competency outcomes (PASS / REVIEW / FAIL)
    - AI narrative summary
    - Candidate feedback
    - Full transcript
    - Recording (if available)

Admins can override the AI hiring recommendation with documented reasoning.

Exporting Reports

Generate PDF reports for hiring committees:

1. Open the interview report
2. Click **Export PDF**
3. Choose sections to include
4. Download report

For Administrators

Team Management

Add team members to your organisation:

1. Go to **Settings** → **Team**
2. Click **Invite Member**
3. Enter email address
4. Assign role:
    - **Admin**: Full access
    - **Recruiter**: Manage roles and interviews
    - **Viewer**: Read-only access
5. Send invitation

Organisation Settings

Configure company-wide settings:

1. Go to **Settings** → **Organisation**
2. Update:
    - Company logo
    - Description
    - Values framework
    - Billing information

Recording Retention

Set how long interview recordings are kept:

1. Go to **Settings** → **Data Retention**
2. Set retention period (default 60 days)
3. Recordings are automatically deleted after this period

Audit Logs

View activity history:

1. Go to **Settings** → **Audit Log**
2. Filter by:
    - Date range
    - User
    - Action type
3. Export for compliance

Frequently Asked Questions

General

Q: Is Talenti free to use? A: Talenti offers both free and paid plans. Contact us for pricing.

Q: What browsers are supported? A: Chrome, Edge, and Firefox. Safari has limited support.

Q: Is my data secure? A: Yes. We use encryption, secure authentication, and follow GDPR guidelines.

For Candidates

Q: Can I redo my interview? A: This depends on the employer's settings. Contact them directly.

Q: How long are interviews? A: Typically 10-15 minutes for AI interviews.

Q: What if I have technical issues? A: Refresh the page. If problems persist, contact support.

Q: Will the AI judge my accent? A: No. The AI evaluates content, not accent or speech patterns.

For Recruiters

Q: How accurate is AI scoring? A: AI scoring provides consistent evaluation. Human review is recommended for final decisions.

Q: Can I edit AI scores? A: Yes. Admins can override scores with documented reasoning.

Q: How do I delete a role? A: Close the role first, then contact support for deletion.

Troubleshooting

Camera/Microphone Not Working

1. Check browser permissions (click lock icon in URL bar)
2. Ensure no other app is using the camera
3. Try a different browser
4. Restart your device

Interview Won't Start

1. Check your internet connection
2. Clear browser cache
3. Disable browser extensions
4. Try incognito/private mode

Invitation Link Expired

Contact the recruiter who sent the invitation to request a new link.

Audio Quality Issues

1. Use headphones to prevent echo
2. Move closer to your microphone
3. Close other applications
4. Check internet bandwidth

Video Freezing

1. Reduce video quality if available
2. Move closer to your router
3. Close background applications
4. Try a wired connection

Login Issues

1. Check email and password
2. Try password reset
3. Check for caps lock
4. Clear cookies and try again

Contact Support

If you need additional help:

- **Email**: support@talenti.app
- **In-App**: Click the help icon in the bottom right
- **Documentation**: docs.talenti.app

Quick Reference Cards

Candidate Quick Start

```
1. Create account → Complete profile → Upload CV
2. Add skills → Add experience → Add education
3. Practice interview → Review feedback
4. Check email for invitations → Complete real interviews
```

Recruiter Quick Start

```
1. Create organisation → Add company details
2. Create job role → Add description → Extract requirements
3. Activate role → Receive applications
4. Generate shortlist → Send invitations
5. Review completed interviews → Make hiring decisions
```